

Another, and probably most devastating argument against user-level threads is that programmers generally want threads in applications where the threads block often, as, for example, in a multithreaded file server. These threads are constantly making system calls. Once a trap has occurred to the kernel to carry out the system call, it is hardly any more work for the kernel to switch threads if the old one has blocked, and having the kernel do this eliminates the need for constantly checking to see if system calls are safe. For applications that are essentially entirely CPU bound and rarely block, what is the point of having threads at all? No one would seriously propose to compute the first  $n$  prime numbers or play chess using threads because there is nothing to be gained by doing it that way.

### Implementing Threads in the Kernel

Now let us consider having the kernel know about and manage the threads. No runtime system is needed, as shown in Fig. 4-8(b). Instead, when a thread wants to create a new thread or destroy an existing thread, it makes a kernel call, which then does the creation or destruction.

To manage all the threads, the kernel has one table per process with one entry per thread. Each entry holds the thread's registers, state, priority, and other information. The information is the same as with user-level threads, but it is now in the kernel instead of in user space (inside the runtime system). This information is also the same information that traditional kernels maintain about each of their single-threaded processes, that is, the process state.

All calls that might block a thread, such as interthread synchronization using semaphores, are implemented as system calls, at considerably greater cost than a call to a runtime system procedure. When a thread blocks, the kernel, at its option, can run either another thread from the same process (if one is ready), or a thread from a different process. With user-level threads, the runtime system keeps running threads from its own process until the kernel takes the CPU away from it (or there are no ready threads left to run).

Due to the relatively greater cost of creating and destroying threads in the kernel, some systems take an environmentally correct approach and recycle their threads. When a thread is destroyed, it is marked as not runnable, but its kernel data structures are not otherwise affected. Later, when a new thread must be created, an old thread is reactivated, saving some overhead. Thread recycling is also possible for user-level threads, but since the thread management overhead is much smaller, there is less incentive to do this.

Kernel threads do not require any new, nonblocking system calls, nor do they lead to deadlocks when spin locks are used. In addition, if one thread in a process causes a page fault, the kernel can easily run another thread while waiting for the required page to be brought in from the disk (or network). Their

main disadvantage is that the cost of a system call is substantial, so if thread operations (creation, deletion, synchronization, etc.) are common, much more overhead will be incurred.

In addition to the various problems specific to user threads and those specific to kernel threads, there are some other problems that occur with both of them. For example, many library procedures are not reentrant. For example, sending a message over the network may well be programmed to assemble the message in a fixed buffer first, then to trap to the kernel to send it. What happens if one thread has assembled its message in the buffer, then a clock interrupt forces a switch to a second thread that immediately overwrites the buffer with its own message? Similarly, after a system call completes, a thread switch may occur before the previous thread has had a chance to read out the error status (*errno*, as discussed above). Also, memory allocation procedures, such as the UNIX *malloc*, fiddle with crucial tables without bothering to set up and use protected critical regions, because they were written for single-threaded environments where that was not necessary. Fixing all these problems properly effectively means rewriting the entire library.

A different solution is to provide each procedure with a jacket that locks a global semaphore or mutex when the procedure is started. In this way, only one thread may be active in the library at once. Effectively, the entire library becomes a big monitor.

Signals also present difficulties. Suppose that one thread wants to catch a particular signal (say, the user hitting the DEL key), and another thread wants this signal to terminate the process. This situation can arise if one or more threads run standard library procedures and others are user-written. Clearly, these wishes are incompatible. In general, signals are difficult enough to manage in a single-threaded environment. Going to a multithreaded environment does not make them any easier to handle. Signals are typically a per-process concept, not a per-thread concept, especially if the kernel is not even aware of the existence of the threads.

### Scheduler Activations

Various researchers have attempted to combine the advantage of user threads (good performance) with the advantage of kernel threads (not having to use a lot of tricks to make things work). Below we will describe one such approach devised by Anderson et al. (1991), called **scheduler activations**. Related work is discussed by Edler et al. (1988) and Scott et al. (1990).

The goals of the scheduler activation work are to mimic the functionality of kernel threads, but with the better performance and greater flexibility usually associated with threads packages implemented in user space. In particular, user

threads should not have to make special nonblocking system calls or check in advance if it is safe to make certain system calls. Nevertheless, when a thread blocks on a system call or on a page fault, it should be possible to run other threads within the same process, if any are ready.

Efficiency is achieved by avoiding unnecessary transitions between user and kernel space. If a thread blocks on a local semaphore, for example, there is no reason to involve the kernel. The user-space runtime system can block the synchronizing thread and schedule a new one by itself.

When scheduler activations are used, the kernel assigns a certain number of virtual processors to each process and lets the (user-space) runtime system allocate threads to processors. This mechanism can also be used on a multiprocessor where the virtual processors may be real CPUs. The number of virtual processors allocated to a process is initially one, but the process can ask for more and can also return processors it no longer needs. The kernel can take back virtual processors already allocated to assign them to other, more needy, processes.

The basic idea that makes this scheme work is that when the kernel knows that a thread has blocked (e.g., by its having executed a blocking system call or caused a page fault), the kernel notifies the process' runtime system, passing as parameters on the stack the number of the thread in question and a description of the event that occurred. The notification happens by having the kernel activate the runtime system at a known starting address, roughly analogous to a signal in UNIX. This mechanism is called an **upcall**.

Once activated like this, the runtime system can reschedule its threads, typically by marking the current thread as blocked and taking another thread from the ready list, setting up its registers, and restarting it. Later, when the kernel learns that the original thread can run again (e.g., the pipe it was trying to read from now contains data, or the page it faulted over has been brought in from disk), it makes another upcall to the runtime system to inform it of this event. The runtime system, at its own discretion, can either restart the blocked thread immediately, or put it on the ready list to be run later.

When a hardware interrupt occurs while a user thread is running, the interrupted CPU switches into kernel mode. If the interrupt is caused by an event not of interest to the interrupted process, such as completion of another process' I/O, when the interrupt handler has finished, it puts the interrupted thread back in the state it was in before the interrupt. If, however, the process is interested in the interrupt, such as the arrival of a page needed by one of the process' threads, the interrupted thread is not restarted. Instead, the interrupted thread is suspended and the runtime system started on that virtual CPU, with the state of the interrupted thread on the stack. It is then up to the runtime system to decide which thread to schedule on that CPU: the interrupted one, the newly ready one, or some third choice.

Although scheduler activations solve the problem of how to pass control to

an unblocked thread in a process one of whose threads has just blocked, it creates a new problem. The new problem is that an interrupted thread might have been executing a semaphore operation at the time it was suspended, in which case it would probably be holding a lock on the ready list. If the runtime system started by the upcall then tries to acquire this lock itself, in order to put a newly ready thread on the list, it will fail to acquire the lock and a deadlock will ensue. The problem can be solved by keeping track of when threads are or are not in critical regions, but the solution is complicated and hardly elegant.

Another objection to scheduler activations is the fundamental reliance on upcalls, a concept that violates the structure inherent in any layered system. Normally, layer  $n$  offers certain services that layer  $n + 1$  can call on, but layer  $n$  may not call procedures in layer  $n + 1$ .

#### 4.1.5. Threads and RPC

It is common for distributed systems to use both RPC and threads. Since threads were invented as a cheap alternative to standard (heavyweight) processes, it is natural that researchers would take a closer look at RPC in this context, to see if it could be made more lightweight as well. In this section we will discuss some interesting work in this area.

Bershad et al. (1990) have observed that even in a distributed system, a substantial number of RPCs are to processes on the same machine as the caller (e.g., to the window manager). Obviously, this result depends on the system, but it is common enough to be worth considering. They have proposed a new scheme that makes it possible for a thread in one process to call a thread in another process on the same machine much more efficiently than the usual way.

The idea works like this. When a server thread,  $S$ , starts up, it exports its interface by telling the kernel about it. The interface defines which procedures are callable, what their parameters are, and so on. When a client thread  $C$  starts up, it imports the interface from the kernel and is given a special identifier to use for the call. The kernel now knows that  $C$  is going to call  $S$  later, and creates special data structures to prepare for the call.

One of these data structures is an argument stack that is shared by both  $C$  and  $S$  and is mapped read/write into both of their address spaces. To call the server,  $C$  pushes the arguments onto the shared stack, using the normal procedure passing conventions, and then traps to the kernel, putting the special identifier in a register. The kernel sees this and knows that the call is local. (If it had been remote, the kernel would have treated the call in the normal manner for remote calls.) It then changes the client's memory map to put the client in the server's address space and starts the client thread executing the server's procedure. The call is made in such a way that the arguments are already in place,

so no copying or marshaling is needed. The net result is that local RPCs can be done much faster this way.

Another technique to speed up RPCs is based on the observation that when a server thread blocks waiting for a new request, it really does not have any important context information. For example, it rarely has any local variables, and there is typically nothing important in its registers. Therefore, when a thread has finished carrying out a request, it simply vanishes and its stack and context information are discarded.

When a new message comes in to the server's machine, the kernel creates a new thread on-the-fly to service the request. Furthermore, it maps the message into the server's address space, and sets up the new thread's stack to access the message. This scheme is sometimes called **implicit receive** and it is in contrast to a conventional thread making a system call to receive a message. The thread that is created spontaneously to handle an incoming RPC is occasionally referred to as a **pop-up thread**. The idea is illustrated in Fig. 4-9.

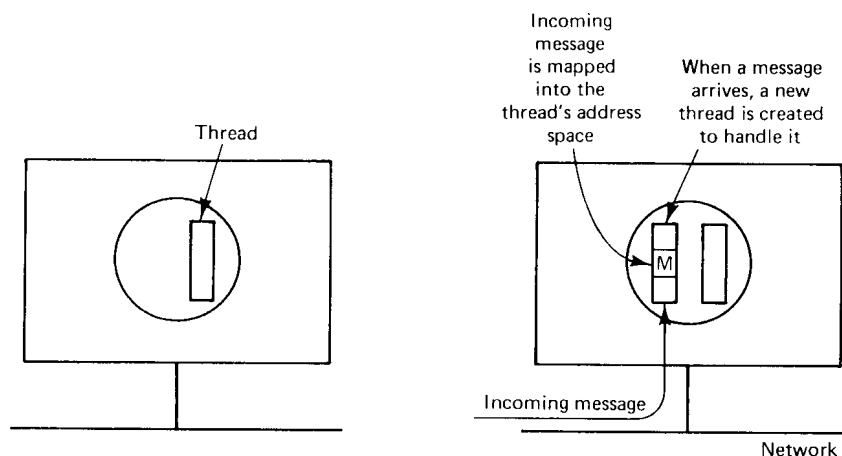


Fig. 4-9. Creating a thread when a message arrives.

The method has several major advantages over conventional RPC. First, threads do not have to block waiting for new work. Thus no context has to be saved. Second, creating a new thread is cheaper than restoring an existing one, since no context has to be restored. Finally, time is saved by not having to copy incoming messages to a buffer within a server thread. Various other techniques can also be used to reduce the overhead. All in all, a substantial gain in speed is possible.

Threads are an ongoing research topic. Some other results are presented in (Marsh et al., 1991; and Draves et al., 1991).

## 4.2. SYSTEM MODELS

Processes run on processors. In a traditional system, there is only one processor, so the question of how the processor should be used does not come up. In a distributed system, with multiple processors, it is a major design issue. The processors in a distributed system can be organized in several ways. In this section we will look at two of the principal ones, the workstation model and the processor pool model, and a hybrid form encompassing features of each one. These models are rooted in fundamentally different philosophies of what a distributed system is all about.

### 4.2.1. The Workstation Model

The workstation model is straightforward: the system consists of workstations (high-end personal computers) scattered throughout a building or campus and connected by a high-speed LAN, as shown in Fig. 4-10. Some of the workstations may be in offices, and thus implicitly dedicated to a single user, whereas others may be in public areas and have several different users during the course of a day. In both cases, at any instant of time, a workstation either has a single user logged into it, and thus has an “owner” (however temporary), or it is idle.

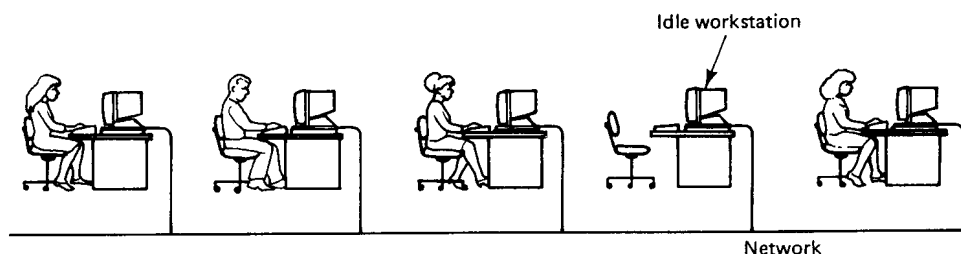


Fig. 4-10. A network of personal workstations, each with a local file system.

In some systems the workstations have local disks and in others they do not. The latter are universally called **diskless workstations**, but the former are variously known as **diskful workstations**, or **disky workstations**, or even stranger names. If the workstations are diskless, the file system must be implemented by one or more remote file servers. Requests to read and write files are sent to a file server, which performs the work and sends back the replies.

Diskless workstations are popular at universities and companies for several reasons, not the least of which is price. Having a large number of workstations equipped with small, slow disks is typically much more expensive than having

one or two file servers equipped with huge, fast disks and accessed over the LAN.

A second reason that diskless workstations are popular is their ease of maintenance. When a new release of some program, say a compiler, comes out, the system administrators can easily install it on a small number of file servers in the machine room. Installing it on dozens or hundreds of machines all over a building or campus is another matter entirely. Backup and hardware maintenance is also simpler with one centrally located 5-gigabyte disk than with fifty 100-megabyte disks scattered over the building.

Another point against disks is that they have fans and make noise. Many people find this noise objectionable and do not want it in their office.

Finally, diskless workstations provide symmetry and flexibility. A user can walk up to any workstation in the system and log in. Since all his files are on the file server, one diskless workstation is as good as another. In contrast, when all the files are stored on local disks, using someone else's workstation means that you have easy access to *his* files, but getting to your own requires extra effort, and is certainly different from using your own workstation.

When the workstations have private disks, these disks can be used in one of at least four ways:

1. Paging and temporary files.
2. Paging, temporary files, and system binaries.
3. Paging, temporary files, system binaries, and file caching.
4. Complete local file system.

The first design is based on the observation that while it may be convenient to keep all the user files on the central file servers (to simplify backup and maintenance, etc.) disks are also needed for paging (or swapping) and for temporary files. In this model, the local disks are used only for paging and files that are temporary, unshared, and can be discarded at the end of the login session. For example, most compilers consist of multiple passes, each of which creates a temporary file read by the next pass. When the file has been read once, it is discarded. Local disks are ideal for storing such files.

The second model is a variant of the first one in which the local disks also hold the binary (executable) programs, such as the compilers, text editors, and electronic mail handlers. When one of these programs is invoked, it is fetched from the local disk instead of from a file server, further reducing the network load. Since these programs rarely change, they can be installed on all the local disks and kept there for long periods of time. When a new release of some system program is available, it is essentially broadcast to all machines. However, if that machine happens to be down when the program is sent, it will miss the

program and continue to run the old version. Thus some administration is needed to keep track of who has which version of which program.

A third approach to using local disks is to use them as explicit caches (in addition to using them for paging, temporaries, and binaries). In this mode of operation, users can download files from the file servers to their own disks, read and write them locally, and then upload the modified ones at the end of the login session. The goal of this architecture is to keep long-term storage centralized, but reduce network load by keeping files local while they are being used. A disadvantage is keeping the caches consistent. What happens if two users download the same file and then each modifies it in different ways? This problem is not easy to solve, and we will discuss it in detail later in the book.

Fourth, each machine can have its own self-contained file system, with the possibility of mounting or otherwise accessing other machines' file systems. The idea here is that each machine is basically self-contained and that contact with the outside world is limited. This organization provides a uniform and guaranteed response time for the user and puts little load on the network. The disadvantage is that sharing is more difficult, and the resulting system is much closer to a network operating system than to a true transparent distributed operating system.

The one diskless and four diskful models we have discussed are summarized in Fig. 4-11. The progression from top to bottom in the figure is from complete dependence on the file servers to complete independence from them.

The advantages of the workstation model are manifold and clear. The model is certainly easy to understand. Users have a fixed amount of dedicated computing power, and thus guaranteed response time. Sophisticated graphics programs can be very fast, since they can have direct access to the screen. Each user has a large degree of autonomy and can allocate his workstation's resources as he sees fit. Local disks add to this independence, and make it possible to continue working to a lesser or greater degree even in the face of file server crashes.

However, the model also has two problems. First, as processor chips continue to get cheaper, it will soon become economically feasible to give each user first 10 and later 100 CPUs. Having 100 workstations in your office makes it hard to see out the window. Second, much of the time users are not using their workstations, which are idle, while other users may need extra computing capacity and cannot get it. From a system-wide perspective, allocating resources in such a way that some users have resources they do not need while other users need these resources badly is inefficient.

The first problem can be addressed by making each workstation a personal multiprocessor. For example, each window on the screen can have a dedicated CPU to run its programs. Preliminary evidence from some early personal multiprocessors such as the DEC Firefly, suggest, however, that the mean number of CPUs utilized is rarely more than one, since users rarely have more than one



|                              | Disk usage                                    | Advantages                                                                 | Disadvantages                                               |
|------------------------------|-----------------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------|
| Dependence on file servers ↑ | (Diskless)                                    | Low cost, easy hardware and software maintenance, symmetry and flexibility | Heavy network usage; file servers may become bottlenecks    |
|                              | Paging, scratch files                         | Reduces network load over diskless case                                    | Higher cost due to large number of disks needed             |
|                              | Paging, scratch files, binaries               | Reduces network load even more                                             | Higher cost; additional complexity of updating the binaries |
|                              | Paging, scratch files, binaries, file caching | Still lower network load; reduces load on file servers as well             | Higher cost; cache consistency problems                     |
|                              | Full local file system                        | Hardly any network load; eliminates need for file servers                  | Loss of transparency                                        |

Fig. 4-11. Disk usage on workstations.

active process at once. Again, this is an inefficient use of resources, but as CPUs get cheaper and cheaper as the technology improves, wasting them will become less of a sin.

#### 4.2.2. Using Idle Workstations

The second problem, idle workstations, has been the subject of considerable research, primarily because many universities have a substantial number of personal workstations, some of which are idle (an idle workstation is the devil's playground?). Measurements show that even at peak periods in the middle of the day, often as many as 30 percent of the workstations are idle at any given moment. In the evening, even more are idle. A variety of schemes have been proposed for using idle or otherwise underutilized workstations (Litzkow et al., 1988; Nichols, 1987; and Theimer et al., 1985). We will describe the general principles behind this work in this section.

The earliest attempt to allow idle workstations to be utilized was the *rsh* program that comes with Berkeley UNIX. This program is called by

```
rsh machine command
```

in which the first argument names a machine and the second names a command to run on it. What *rsh* does is run the specified command on the specified machine. Although widely used, this program has several serious flaws. First,

the user must tell which machine to use, putting the full burden of keeping track of idle machines on the user. Second, the program executes in the environment of the remote machine, which is usually different from the local environment. Finally, if someone should log into an idle machine on which a remote process is running, the process continues to run and the newly logged-in user either has to accept the lower performance or find another machine.

The research on idle workstations has centered on solving these problems. The key issues are:

1. How is an idle workstation found?
2. How can a remote process be run transparently?
3. What happens if the machine's owner comes back?

Let us consider these three issues, one at a time.

How is an idle workstation found? To start with, what is an idle workstation? At first glance, it might appear that a workstation with no one logged in at the console is an idle workstation, but with modern computer systems things are not always that simple. In many systems, even with no one logged in there may be dozens of processes running, such as clock daemons, mail daemons, news daemons, and all manner of other daemons. On the other hand, a user who logs in when arriving at his desk in the morning, but otherwise does not touch the computer for hours, hardly puts any additional load on it. Different systems make different decisions as to what "idle" means, but typically, if no one has touched the keyboard or mouse for several minutes and no user-initiated processes are running, the workstation can be said to be idle. Consequently, there may be substantial differences in load between one idle workstation and another, due, for example, to the volume of mail coming into the first one but not the second.

The algorithms used to locate idle workstations can be divided into two categories: server driven and client driven. In the former, when a workstation goes idle, and thus becomes a potential compute server, it announces its availability. It can do this by entering its name, network address, and properties in a registry file (or data base), for example. Later, when a user wants to execute a command on an idle workstation, he types something like

```
remote command
```

and the *remote* program looks in the registry to find a suitable idle workstation. For reliability reasons, it is also possible to have multiple copies of the registry.

An alternative way for the newly idle workstation to announce the fact that it has become unemployed is to put a broadcast message onto the network. All other workstations then record this fact. In effect, each machine maintains its

own private copy of the registry. The advantage of doing it this way is less overhead in finding an idle workstation and greater redundancy. The disadvantage is requiring all machines to do the work of maintaining the registry.

Whether there is one registry or many, there is a potential danger of race conditions occurring. If two users invoke the *remote* command simultaneously, and both of them discover that the same machine is idle, they may both try to start up processes there at the same time. To detect and avoid this situation, the *remote* program can check with the idle workstation, which, if still free, removes itself from the registry and gives the go-ahead sign. At this point, the caller can send over its environment and start the remote process, as shown in Fig. 4-12.

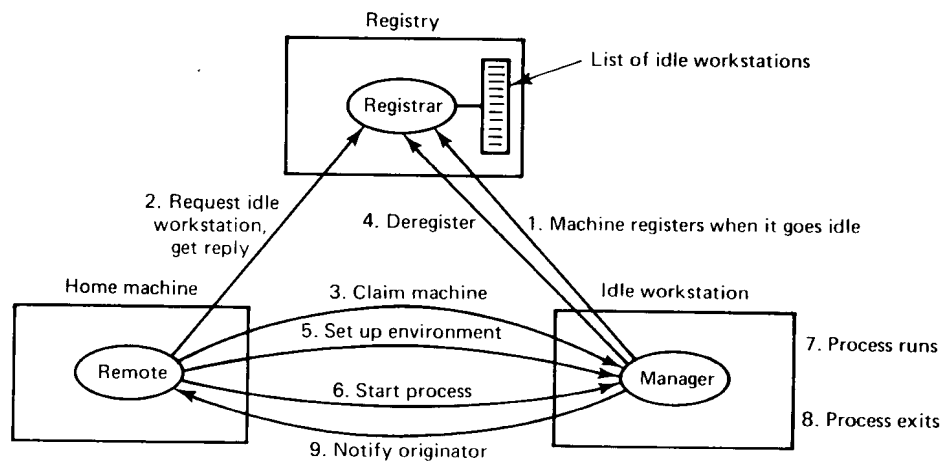


Fig. 4-12. A registry-based algorithm for finding and using idle workstations.

The other way to locate idle workstations is to use a client-driven approach. When *remote* is invoked, it broadcasts a request saying what program it wants to run, how much memory it needs, whether or not floating point is needed, and so on. These details are not needed if all the workstations are identical, but if the system is heterogeneous and not every program can run on every workstation, they are essential. When the replies come back, *remote* picks one and sets it up. One nice twist is to have “idle” workstations delay their responses slightly, with the delay being proportional to the current load. In this way, the reply from the least heavily loaded machine will come back first and be selected.

Finding a workstation is only the first step. Now the process has to be run there. Moving the code is easy. The trick is to set up the remote process so that it sees the same environment it would have locally, on the **home workstation**, and thus carries out the same computation it would have locally.

To start with, it needs the same view of the file system, the same working directory, and the same environment variables (shell variables), if any. After

these have been set up, the program can begin running. The trouble starts when the first system call, say a READ, is executed. What should the kernel do? The answer depends very much on the system architecture. If the system is diskless, with all the files located on file servers, the kernel can just send the request to the appropriate file server, the same way the home machine would have done had the process been running there. On the other hand, if the system has local disks, each with a complete file system, the request has to be forwarded back to the home machine for execution.

Some system calls must be forwarded back to the home machine no matter what, even if all the machines are diskless. For example, reads from the keyboard and writes to the screen can never be carried out on the remote machine. However, other system calls must be done remotely under all conditions. For example, the UNIX system calls SBRK (adjust the size of the data segment), NICE (set CPU scheduling priority), and PROFIL (enable profiling of the program counter) cannot be executed on the home machine. In addition, all system calls that query the state of the machine have to be done on the machine on which the process is actually running. These include asking for the machine's name and network address, asking how much free memory it has, and so on.

System calls involving time are a problem because the clocks on different machines may not be synchronized. In Chap. 3, we saw how hard it is to achieve synchronization. Using the time on the remote machine may cause programs that depend on time, like *make*, to give incorrect results. Forwarding all time-related calls back to the home machine, however, introduces delay, which also causes problems with time.

To complicate matters further, certain special cases of calls which normally might have to be forwarded back, such as creating and writing to a temporary file, can be done much more efficiently on the remote machine. In addition, mouse tracking and signal propagation have to be thought out carefully as well. Programs that write directly to hardware devices, such as the screen's frame buffer, diskette, or magnetic tape, cannot be run remotely at all. All in all, making programs run on remote machines as though they were running on their home machines is possible, but it is a complex and tricky business.

The final question on our original list is what to do if the machine's owner comes back (i.e., somebody logs in or a previously inactive user touches the keyboard or mouse). The easiest thing is to do nothing, but this tends to defeat the idea of "personal" workstations. If other people can run programs on your workstation at the same time that you are trying to use it, there goes your guaranteed response.

Another possibility is to kill off the intruding process. The simplest way is to do this abruptly and without warning. The disadvantage of this strategy is that all work will be lost and the file system may be left in a chaotic state. A better way is to give the process fair warning, by sending it a signal to allow it to

detect impending doom, and shut down gracefully (write edit buffers to the disk, close files, and so on). If it has not exited within a few seconds, it is then terminated. Of course, the program must be written to expect and handle this signal, something most existing programs definitely are not.

A completely different approach is to migrate the process to another machine, either back to the home machine or to yet another idle workstation. Migration is rarely done in practice because the actual mechanism is complicated. The hard part is not moving the user code and data, but finding and gathering up all the kernel data structures relating to the process that is leaving. For example, it may have open files, running timers, queued incoming messages, and other bits and pieces of information scattered around the kernel. These must all be carefully removed from the source machine and successfully reinstalled on the destination machine. There are no theoretical problems here, but the practical engineering difficulties are substantial. For more information, see (Artsy and Finkel, 1989; Douglass and Ousterhout, 1991; and Zayas, 1987).

In both cases, when the process is gone, it should leave the machine in the same state in which it found it, to avoid disturbing the owner. Among other items, this requirement means that not only must the process go, but also all its children and their children. In addition, mailboxes, network connections, and other system-wide data structures must be deleted, and some provision must be made to ignore RPC replies and other messages that arrive for the process after it is gone. If there is a local disk, temporary files must be deleted, and if possible, any files that had to be removed from its cache restored.

#### 4.2.3. The Processor Pool Model

Although using idle workstations adds a little computing power to the system, it does not address a more fundamental issue: What happens when it is feasible to provide 10 or 100 times as many CPUs as there are active users? One solution, as we saw, is to give everyone a personal multiprocessor. However this is a somewhat inefficient design.

An alternative approach is to construct a **processor pool**, a rack full of CPUs in the machine room, which can be dynamically allocated to users on demand. The processor pool approach is illustrated in Fig. 4-13. Instead of giving users personal workstations, in this model they are given high-performance graphics terminals, such as X terminals (although small workstations can also be used as terminals). This idea is based on the observation that what many users really want is a high-quality graphical interface and good performance. Conceptually, it is much closer to traditional timesharing than to the personal computer model, although it is built with modern technology (low-cost microprocessors).

The motivation for the processor pool idea comes from taking the diskless workstation idea a step further. If the file system can be centralized in a small

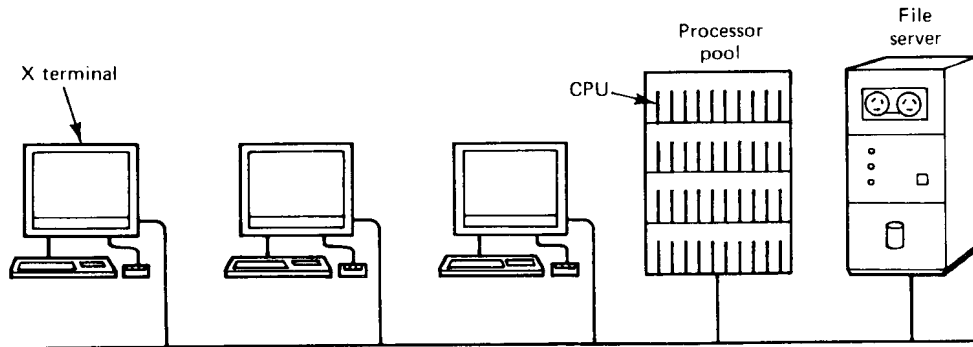


Fig. 4-13. A system based on the processor pool model.

number of file servers to gain economies of scale, it should be possible to do the same thing for compute servers. By putting all the CPUs in a big rack in the machine room, power supply and other packaging costs can be reduced, giving more computing power for a given amount of money. Furthermore, it permits the use of cheaper X terminals (or even ordinary ASCII terminals), and decouples the number of users from the number of workstations. The model also allows for easy incremental growth. If the computing load increases by 10 percent, you can just buy 10 percent more processors and put them in the pool.

In effect, we are converting all the computing power into “idle workstations” that can be accessed dynamically. Users can be assigned as many CPUs as they need for short periods, after which they are returned to the pool so that other users can have them. There is no concept of ownership here: all the processors belong equally to everyone.

The biggest argument for centralizing the computing power in a processor pool comes from queueing theory. A queueing system is a situation in which users generate random requests for work from a server. When the server is busy, the users queue for service and are processed in turn. Common examples of queueing systems are bakeries, airport check-in counters, supermarket check-out counters, and numerous others. The bare basics are depicted in Fig. 4-14.

Queueing systems are useful because it is possible to model them analytically. Let us call the total input rate  $\lambda$  requests per second, from all the users combined. Let us call the rate at which the server can process requests  $\mu$ . For stable operation, we must have  $\mu > \lambda$ . If the server can handle 100 requests/sec, but the users continuously generate 110 requests/sec, the queue will grow without bound. (Small intervals in which the input rate exceeds the service rate are acceptable, provided that the mean input rate is lower than the service rate and there is enough buffer space.)

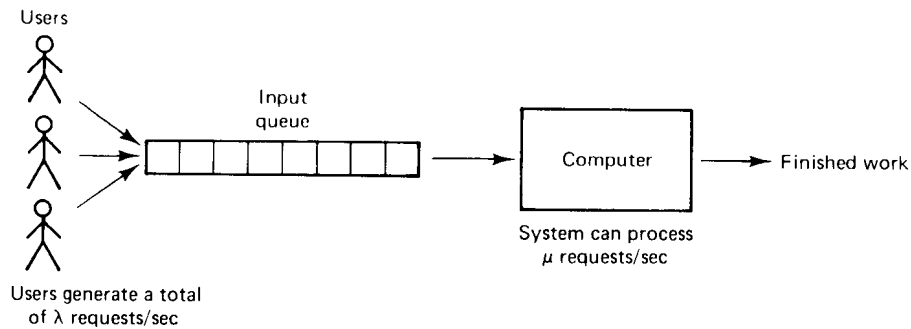


Fig. 4-14. A basic queueing system.

It can be proven (Kleinrock, 1974) that the mean time between issuing a request and getting a complete response,  $T$ , is related to  $\lambda$  and  $\mu$  by the formula

$$T = \frac{1}{\mu - \lambda}$$

As an example, consider a file server that is capable of handling as many as 50 requests/sec but which only gets 40 requests/sec. The mean response time will be 1/10 sec or 100 msec. Note that when  $\lambda$  goes to 0 (no load), the response time of the file server does not go to 0, but to 1/50 sec or 20 msec. The reason is obvious once it is pointed out. If the file server can process only 50 requests/sec, it must take 20 msec to process a single request, even in the absence of any competition, so the response time, which includes the processing time, can never go below 20 msec.

Suppose that we have  $n$  personal multiprocessors, each with some number of CPUs, and each one forms a separate queueing system with request arrival rate  $\lambda$  and CPU processing rate  $\mu$ . The mean response time,  $T$ , will be as given above. Now consider what happens if we scoop up all the CPUs and place them in a single processor pool. Instead of having  $n$  small queueing systems running in parallel, we now have one large one, with an input rate  $n\lambda$  and a service rate  $n\mu$ . Let us call the mean response time of this combined system  $T_1$ . From the formula above we find

$$T_1 = \frac{1}{n\mu - n\lambda} = T/n$$

This surprising result says that by replacing  $n$  small resources by one big one that is  $n$  times more powerful, we can reduce the average response time  $n$ -fold.

This result is extremely general and applies to a large variety of systems. It

is one of the main reasons that airlines prefer to fly a 300-seat 747 once every 5 hours to a 10-seat business jet every 10 minutes. The effect arises because dividing the processing power into small servers (e.g., personal workstations), each with one user, is a poor match to a workload of randomly arriving requests. Much of the time, a few servers are busy, even overloaded, but most are idle. It is this wasted time that is eliminated in the processor pool model, and the reason why it gives better overall performance. The concept of using idle workstations is a weak attempt at recapturing the wasted cycles, but it is complicated and has many problems, as we have seen.

In fact, this queueing theory result is one of the main arguments against having distributed systems at all. Given a choice between one centralized 1000-MIPS CPU and 100 private, dedicated, 10-MIPS CPUs, the mean response time of the former will be 100 times better, because no cycles are ever wasted. The machine goes idle only when no user has any work to do. This fact argues in favor of concentrating the computing power as much as possible.

However, mean response time is not everything. There are also arguments in favor of distributed computing, such as cost. If a single 1000-MIPS CPU is much more expensive than 100 10-MIPS CPUs, the price/performance ratio of the latter may be much better. It may not even be possible to build such a large machine at any price. Reliability and fault tolerance are also factors.

Also, personal workstations have a uniform response, independent of what other people are doing (except when the network or file servers are jammed). For some users, a low variance in response time may be perceived as more important than the mean response time itself. Consider, for example, editing on a private workstation on which asking for the next page to be displayed always takes 500 msec. Now consider editing on a large, centralized, shared computer on which asking for the next page takes 5 msec 95 percent of the time and 5 sec one time in 20. Even though the mean here is twice as good as on the workstation, the users may consider the performance intolerable. On the other hand, to the user with a huge simulation to run, the big computer may win hands down.

So far we have tacitly assumed that a pool of  $n$  processors is effectively the same thing as a single processor that is  $n$  times as fast as a single processor. In reality, this assumption is justified only if all requests can be split up in such a way as to allow them to run on all the processors in parallel. If a job can be split into, say, only 5 parts, then the processor pool model has an effective service time only 5 times better than that of a single processor, not  $n$  times better.

Still, the processor pool model is a much cleaner way of getting extra computing power than looking around for idle workstations and sneaking over there while nobody is looking. By starting out with the assumption that no processor belongs to anyone, we get a design based on the concept of requesting machines from the pool, using them, and putting them back when done. There is also no need to forward anything back to a “home” machine because there are none.



There is also no danger of the owner coming back, because there are no owners.

In the end, it all comes down to the nature of the workload. If all people are doing is simple editing and occasionally sending an electronic mail message or two, having a personal workstation is probably enough. If, on the other hand, the users are engaged in a large software development project, frequently running *make* on large directories, or are trying to invert massive sparse matrices, or do major simulations or run big artificial intelligence or VLSI routing programs, constantly hunting for substantial numbers of idle workstations will be no fun at all. In all these situations, the processor pool idea is fundamentally much simpler and more attractive.

#### 4.2.4. A Hybrid Model

A possible compromise is to provide each user with a personal workstation and to have a processor pool in addition. Although this solution is more expensive than either a pure workstation model or a pure processor pool model, it combines the advantages of both of the others.

Interactive work can be done on workstations, giving guaranteed response. Idle workstations, however, are not utilized, making for a simpler system design. They are just left unused. Instead, all noninteractive processes run on the processor pool, as does all heavy computing in general. This model provides fast interactive response, an efficient use of resources, and a simple design.

### 4.3. PROCESSOR ALLOCATION

By definition, a distributed system consists of multiple processors. These may be organized as a collection of personal workstations, a public processor pool, or some hybrid form. In all cases, some algorithm is needed for deciding which process should be run on which machine. For the workstation model, the question is when to run a process locally and when to look for an idle workstation. For the processor pool model, a decision must be made for every new process. In this section we will study the algorithms used to determine which process is assigned to which processor. We will follow tradition and refer to this subject as “processor allocation” rather than “process allocation,” although a good case can be made for the latter.

#### 4.3.1. Allocation Models

Before looking at specific algorithms, or even at design principles, it is worthwhile saying something about the underlying model, assumptions, and goals of the work on processor allocation. Nearly all work in this area assumes

that all the machines are identical, or at least code-compatible, differing at most by speed. An occasional paper assumes that the system consists of several disjoint processor pools, each of which is homogeneous. These assumptions are usually valid, and make the problem much simpler, but leave unanswered for the time being such questions as whether a command to start up the text formatter should be started up on a 486, SPARC, or MIPS CPU, assuming that binaries for all of them are available.

Almost all published models assume that the system is fully interconnected, that is, every processor can communicate with every other processor. We will assume this as well. This assumption does not mean that every machine has a wire to every other machine, just that transport connections can be established between every pair. That messages may have to be routed hop by hop over a sequence of machines is of interest only to the lower layers. Some networks support broadcasting or multicasting, and some algorithms use these facilities.

New work is generated when a running process decides to fork or otherwise create a subprocess. In some cases the forking process is the command interpreter (shell) that is starting up a new job in response to a command from the user. In others, a user process itself creates one or more children, for example, in order to gain performance by having all the children run in parallel.

Processor allocation strategies can be divided into two broad classes. In the first, which we shall call **nonmigratory**, when a process is created, a decision is made about where to put it. Once placed on a machine, the process stays there until it terminates. It may not move, no matter how badly overloaded its machine becomes and no matter how many other machines are idle. In contrast, with **migratory** allocation algorithms, a process can be moved even if it has already started execution. While migratory strategies allow better load balancing, they are more complex and have a major impact on system design.

Implicit in an algorithm that assigns processes to processors is that we are trying to optimize something. If this were not the case, we could just make the assignments at random or in numerical order. Precisely what it is that is being optimized, however, varies from one system to another. One possible goal is to maximize **CPU utilization**, that is, maximize the number of CPU cycles actually executed on behalf of user jobs per hour of real time. Maximizing CPU utilization is another way of saying that CPU idle time is to be avoided at all costs. When in doubt, make sure that every CPU has something to do.

Another worthy objective is minimizing mean **response time**. Consider, for example, the two processors and two processes of Fig. 4-15. Processor 1 runs at 10 MIPS; processor 2 runs at 100 MIPS, but has a waiting list of backlogged processes that will take 5 sec to finish off. Process *A* has 100 million instructions and process *B* has 300 million. The response times for each process on each processor (including the wait time) are shown in the figure. If we assign *A* to processor 1 and *B* to processor 2, the mean response time will be  $(10 + 8)/2$

or 9 sec. If we assign them the other way around, the mean response time will be  $(30 + 6)/2$  or 18 sec. Clearly, the former is a better assignment in terms of minimizing mean response time.

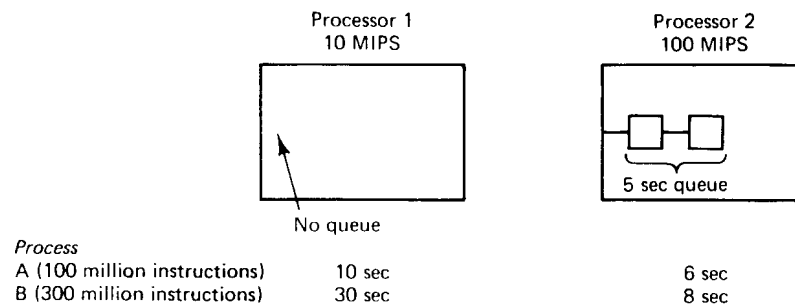


Fig. 4-15. Response times of two processes on two processors.

A variation of minimizing the response time is minimizing the **response ratio**. The response ratio is defined as the amount of time it takes to run a process on some machine, divided by how long it would take on some unloaded benchmark processor. For many users, response ratio is a more useful metric than response time since it takes into account the fact that big jobs are supposed to take longer than small ones. To see this point, which is better, a 1-sec job that takes 5 sec or a 1-min job that takes 70 sec? Using response time, the former is better, but using response ratio, the latter is much better because  $5/1 \gg 70/60$ .

#### 4.3.2. Design Issues for Processor Allocation Algorithms

A large number of processor allocation algorithms have been proposed over the years. In this section we will look at some of the key choices involved in these algorithms and point out the various trade-offs. The major decisions the designers must make can be summed up in five issues:

1. Deterministic versus heuristic algorithms.
2. Centralized versus distributed algorithms.
3. Optimal versus suboptimal algorithms.
4. Local versus global algorithms.
5. Sender-initiated versus receiver-initiated algorithms.

Other decisions also come up, but these are the main ones that have been studied extensively in the literature. Let us look at each of these in turn.

Deterministic algorithms are appropriate when everything about process behavior is known in advance. Imagine that you have a complete list of all processes, their computing requirements, their file requirements, their communication requirements, and so on. Armed with this information, it is possible to make a perfect assignment. In theory, one could try all possible assignments and take the best one.

In few, if any, systems, is total knowledge available in advance, but sometimes a reasonable approximation is obtainable. For example, in banking, insurance, or airline reservations, today's work is just like yesterday's. The airlines have a pretty good idea of how many people want to fly from New York to Chicago on a Monday morning in early Spring, so the nature of the workload can be accurately characterized, at least statistically, making it possible to consider deterministic allocation algorithms.

At the other extreme are systems where the load is completely unpredictable. Requests for work depend on who's doing what, and can change dramatically from hour to hour, or even from minute to minute. Processor allocation in such systems cannot be done in a deterministic, mathematical way, but of necessity uses ad hoc techniques called **heuristics**.

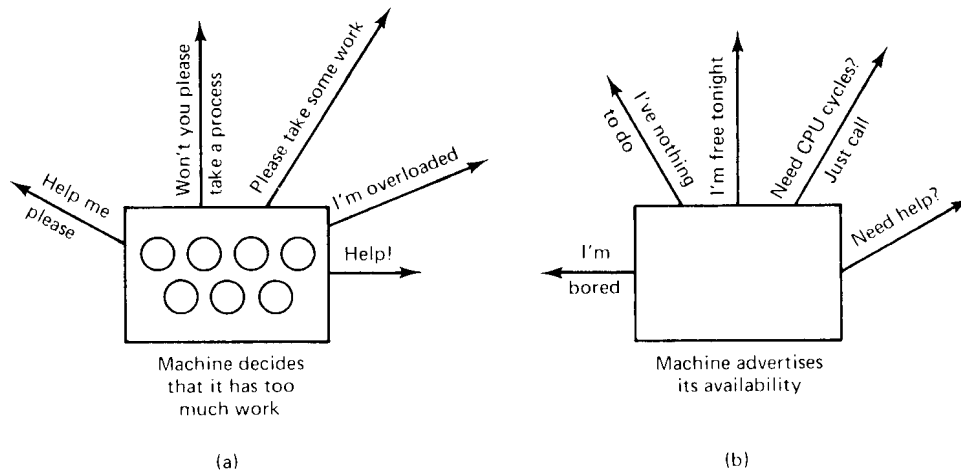
The second design issue is centralized versus distributed. This theme has occurred repeatedly throughout the book. Collecting all the information in one place allows a better decision to be made, but is less robust and can put a heavy load on the central machine. Decentralized algorithms are usually preferable, but some centralized algorithms have been proposed for lack of suitable decentralized alternatives.

The third issue is related to the first two: Are we trying to find the best allocation, or merely an acceptable one? Optimal solutions can be obtained in both centralized and decentralized systems, but are invariably more expensive than suboptimal ones. They involve collecting more information and processing it more thoroughly. In practice, most actual distributed systems settle for heuristic, distributed, suboptimal solutions because it is hard to obtain optimal ones.

The fourth issue relates to what is often called **transfer policy**. When a process is about to be created, a decision has to be made whether or not it can be run on the machine where it is being generated. If that machine is too busy, the new process must be transferred somewhere else. The choice here is whether or not to base the transfer decision entirely on local information. One school of thought advocates a simple (local) algorithm: if the machine's load is below some threshold, keep the new process; otherwise, try to get rid of it. Another school says that this heuristic is too crude. Better to collect (global) information about the load elsewhere before deciding whether or not the local machine is too busy for another process. Each school has its points. Local algorithms are simple, but may be far from optimal, whereas global ones may only give a slightly better result at much higher cost.

The last issue in our list deals with **location policy**. Once the transfer policy has decided to get rid of a process, the location policy has to figure out where to send it. Clearly, the location policy cannot be local. It needs information about the load elsewhere to make an intelligent decision. This information can be disseminated in two ways, however. In one method, the senders start the information exchange. In another, it is the receivers that take the initiative.

As a simple example, look at Fig. 4-16(a). Here an overloaded machine sends out requests for help to other machines, in hopes of offloading its new process on some other machine. The sender takes the initiative in locating more CPU cycles in this example. In contrast, in Fig. 4-16(b), a machine that is idle or underloaded announces to other machines that it has little to do and is prepared to take on extra work. Its goal is to locate a machine that is willing to give it some work to do.



**Fig. 4-16.** (a) A sender looking for an idle machine. (b) A receiver looking for work to do.

For both the sender-initiated and receiver-initiated cases, various algorithms have different strategies for whom to probe, how long to continue probing, and what to do with the results. Nevertheless, the difference between the two approaches should be clear by now.

#### 4.3.3. Implementation Issues for Processor Allocation Algorithms

The points raised in the preceding section are all clear-cut theoretical issues about which one can have endless wonderful debates. In this section we will look at some other issues that are more related to the nitty-gritty details of

implementing processor allocation algorithms than to the great principles behind them.

To start with, virtually all the algorithms assume that machines know their own load, so they can tell if they are underloaded or overloaded, and can tell other machines about their state. Measuring load is not as simple as it first appears. One approach is simply to count the number of processes on each machine and use that number as the load. However, as we have pointed out before, even on an idle system there may be many processes running, including mail and news daemons, window managers, and other processes. Thus the process count says almost nothing about the current load.

The next step is to count only processes that are running or ready. After all, every running or runnable process puts some load on the machine, even if it is a background process. However, many of these daemons wake up periodically, check to see if anything interesting has happened, and if not, go back to sleep. Most put only a small load on the system.

A more direct measurement, although it is more work to capture, is the fraction of time the CPU is busy. Clearly, a machine with a 20 percent CPU utilization is more heavily loaded than one with a 10 percent CPU utilization, whether it is running user or daemon programs. One way to measure the CPU utilization is to set up a timer and let it interrupt the machine periodically. At each interrupt, the state of the CPU is observed. In this way, the fraction of time spent in the idle loop can be observed.

A problem with timer interrupts is that when the kernel is executing critical code, it will often disable all interrupts, including the timer interrupt. Thus if the timer goes off while the kernel is active, the interrupt will be delayed until the kernel finishes. If the kernel was in the process of blocking the last active processes, the timer will not go off until the kernel has finished—and entered the idle loop. This effect will tend to underestimate the true CPU usage.

Another implementation issue is how overhead is dealt with. Many theoretical processor allocation algorithms ignore the overhead of collecting measurements and moving processes around. If an algorithm discovers that by moving a newly created process to a distant machine it can improve system performance by 10 percent, it may be better to do nothing, since the cost of moving the process may eat up all the gain. A proper algorithm should take into account the CPU time, memory usage, and network bandwidth consumed by the processor allocation algorithm itself. Few do, mostly because it is not easy.

Our next implementation consideration is complexity. Virtually all researchers measure the quality of their algorithms by looking at analytical, simulation, or experimental measures of CPU utilization, network usage, and response time. Seldom is the complexity of the software considered, despite the obvious implications for system performance, correctness, and robustness. It rarely happens that someone publishes a new algorithm, demonstrates how good

its performance is, and then concludes that the algorithm is not worth using because its performance is only slightly better than existing algorithms but is much more complicated to implement (or slower to run).

In this vein, a study by Eager et al. (1986) sheds light on the subject of pursuing complex, optimal algorithms. They studied three algorithms. In all cases, each machine measures its own load and decides for itself whether it is underloaded. Whenever a new process is created, the creating machine checks to see if it is overloaded. If so, it seeks out a remote machine on which to start the new process. The three algorithms differ in how the candidate machine is located.

Algorithm 1 picks a machine at random and just sends the new process there. If the receiving machine itself is overloaded, it, too, picks a random machine and sends the process off. This process is repeated until either somebody is willing to take it, or a hop counter is exceeded, in which case no more forwarding is permitted.

Algorithm 2 picks a machine at random and sends it a probe asking if it is underloaded or overloaded. If the machine admits to being underloaded, it gets the new process; otherwise, a new probe is tried. This loop is repeated until a suitable machine is found or the probe limit is exceeded, in which case it stays where it is created.

Algorithm 3 probes  $k$  machines to determine their exact loads. The process is then sent to the machine with the smallest load.

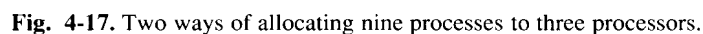
Intuitively, if we ignore all the overhead of the probes and process transfers, one would expect algorithm 3 to have the best performance, and indeed it does. But the gain in performance of algorithm 3 over algorithm 2 is small, even though the complexity and amount of additional work required are larger. Eager et al. concluded that if using a simple algorithm gives you most of the gain of a much more expensive and complicated one, it is better to use the simple one.

Our final point here is that stability is also an issue that crops up. Different machines run their algorithms asynchronously from one another, so the system is rarely in equilibrium. It is possible to get into situations where neither  $A$  nor  $B$  has quite up-to-date information, and each thinks the other has a lighter load, resulting in some poor process being shuttled back and forth repeatedly. The problem is that most algorithms that exchange information can be shown to be correct after all the information has been exchanged and everything has settled down, but little can be said about their operation while tables are still being updated. It is in these nonequilibrium situations that problems often arise.

#### 4.3.4. Example Processor Allocation Algorithms

To provide insight into how processor allocation can really be accomplished, in this section we will discuss several different algorithms. These have been selected to cover a broad range of possibilities, but there are others as well.

The system can be represented as a weighted graph, with each node being a process and each arc representing the flow of messages between two processes. Mathematically, the problem then reduces to finding a way to partition (i.e., cut) the graph into  $k$  disjoint subgraphs, subject to certain constraints (e.g., total CPU and memory requirements below some limits for each subgraph). For each solution that meets the constraints, arcs that are entirely within a single subgraph represent intramachine communication and can be ignored. Arcs that go from one subgraph to another represent network traffic. The goal is then to find the partitioning that minimizes the network traffic while meeting all the constraints. Figure 4-17 shows two ways of partitioning the same graph, yielding two different network loads.



Intuitively, what we are doing is looking for clusters that are tightly coupled (high intracluster traffic flow) but which interact little with other clusters (low intercluster traffic flow). Some of the many papers discussing the problem are (Chow and Abraham, 1982; Stone and Bokhari, 1978; and Lo, 1984).



### A Centralized Algorithm

Graph-theoretic algorithms of the kind we have just discussed are of limited applicability since they require complete information in advance, so let us turn to a heuristic algorithm that does not require any advance information. This algorithm, called **up-down** (Mutka and Livny, 1987), is centralized in the sense that a coordinator maintains a **usage table** with one entry per personal workstation (i.e., per user), initially zero. When significant events happen, messages are sent to the coordinator to update the table. Allocation decisions are based on the table. These decisions are made when scheduling events happen: a processor is being requested, a processor has become free, or the clock has ticked.

The unusual thing about this algorithm, and the reason that it is centralized, is that instead of trying to maximize CPU utilization, it is concerned with giving each workstation owner a fair share of the computing power. Whereas other algorithms will happily let one user take over all the machines if he promises to keep them all busy (i.e., achieve a high CPU utilization), this algorithm is designed to prevent precisely that.

When a process is to be created, and the machine it is created on decides that the process should be run elsewhere, it asks the usage table coordinator to allocate it a processor. If there is one available and no one else wants it, the request is granted. If no processors are free, the request is temporarily denied and a note is made of the request.

When a workstation owner is running processes on other people's machines, it accumulates penalty points, a fixed number per second, as shown in Fig. 4-18. These points are added to its usage table entry. When it has unsatisfied requests pending, penalty points are subtracted from its usage table entry. When no requests are pending and no processors are being used, the usage table entry is moved a certain number of points closer to zero, until it gets there. In this way, the score goes up and down, hence the name of the algorithm.

Usage table entries can be positive, zero, or negative. A positive score indicates that the workstation is a net user of system resources, whereas a negative score means that it needs resources. A zero score is neutral.

The heuristic used for processor allocation can now be given. When a processor becomes free, the pending request whose owner has the lowest score wins. As a consequence, a user who is occupying no processors and who has had a request pending for a long time will always beat someone who is using many processors. This property is the intention of the algorithm, to allocate capacity fairly.

In practice this means that if one user has a fairly continuous load on the system, and another user comes along and wants to start a process, the light user will be favored over the heavy one. Simulation studies (Mutka and Livny, 1987) show that the algorithm works as expected under a variety of load conditions.

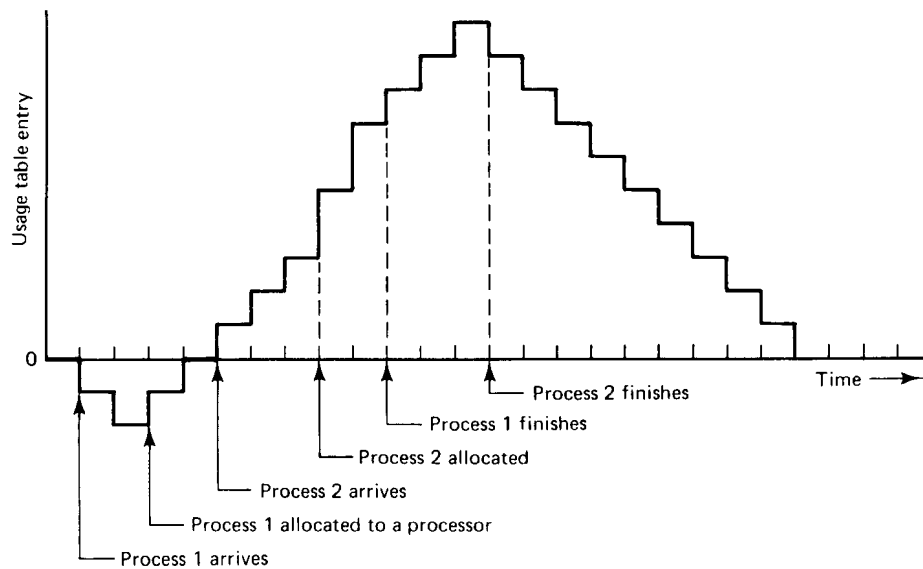


Fig. 4-18. Operation of the up-down algorithm.

### A Hierarchical Algorithm

Centralized algorithms, such as up-down, do not scale well to large systems. The central node soon becomes a bottleneck, not to mention a single point of failure. These problems can be attacked by using a hierarchical algorithm instead of a centralized one. Hierarchical algorithms retain much of the simplicity of centralized ones, but scale better.

One approach that has been proposed for keeping tabs on a collection of processors is to organize them in a logical hierarchy independent of the physical structure of the network, as in MICROS (Wittie and van Tilborg, 1980). This approach organizes the machines like people in corporate, military, academic, and other real-world hierarchies. Some of the machines are workers and others are managers.

For each group of  $k$  workers, one manager machine (the “department head”) is assigned the task of keeping track of who is busy and who is idle. If the system is large, there will be an unwieldy number of department heads, so some machines will function as “deans,” each riding herd on some number of department heads. If there are many deans, they too can be organized hierarchically, with a “big cheese” keeping tabs on a collection of deans. This hierarchy can be extended ad infinitum, with the number of levels needed growing logarithmically with the number of workers. Since each processor need only maintain

communication with one superior and a few subordinates, the information stream is manageable.

An obvious question is: What happens when a department head, or worse yet, a big cheese, stops functioning (crashes)? One answer is to promote one of the direct subordinates of the faulty manager to fill in for the boss. The choice of which can be made by the subordinates themselves, by the deceased's peers, or in a more autocratic system, by the sick manager's boss.

To avoid having a single (vulnerable) manager at the top of the tree, one can truncate the tree at the top and have a committee as the ultimate authority, as shown in Fig. 4-19. When a member of the ruling committee malfunctions, the remaining members promote someone one level down as replacement.

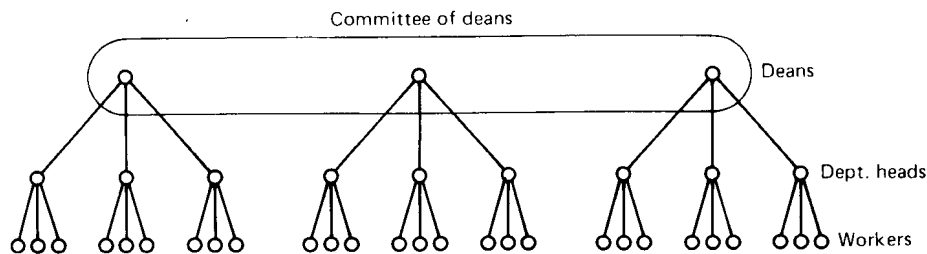


Fig. 4-19. A processor hierarchy can be modeled as an organizational hierarchy.

While this scheme is not really distributed, it is feasible, and in practice works well. In particular, the system is self-repairing and can survive occasional crashes of both workers and managers without any long-term effects.

In MICROS, the processors are monoprogrammed, so if a job requiring  $S$  processes suddenly appears, the system must allocate  $S$  processors for it. Jobs can be created at any level of the hierarchy. The strategy used is for each manager to keep track of approximately how many workers below it are available (possibly several levels below it). If it thinks that a sufficient number are available, it reserves some number  $R$  of them, where  $R \geq S$ , because the estimate of available workers may not be exact and some machines may be down.

If the manager receiving the request thinks that it has too few processors available, it passes the request upward in the tree to its boss. If the boss cannot handle it either, the request continues propagating upward until it reaches a level that has enough available workers at its disposal. At that point, the manager splits the request into parts and parcels them out among the managers below it, which then do the same thing until the wave of allocation requests hits bottom. At the bottom level, the processors are marked as "busy" and the actual number of processors allocated is reported back up the tree.

To make this strategy work well,  $R$  must be large enough that the probability is high that enough workers will be found to handle the entire job. Otherwise

the request will have to move up one level in the tree and start all over, wasting considerable time and computing power. On the other hand, if  $R$  is too large, too many processors will be allocated, wasting computing capacity until word gets back to the top and they can be released.

The whole situation is greatly complicated by the fact that requests for processors can be generated randomly anywhere in the system, so at any instant, multiple requests are likely to be in various stages of the allocation algorithm, potentially giving rise to out-of-date estimates of available workers, race conditions, deadlocks, and more. In Van Tilborg and Wittie (1981) a mathematical analysis of the problem is given and various other aspects not described here are covered in detail.

### **A Sender-Initiated Distributed Heuristic Algorithm**

The algorithms described above are all centralized or semicentralized. Distributed algorithms also exist. Typical of these are the ones described by Eager et al. (1986). As mentioned above, in the most cost-effective algorithm they studied, when a process is created, the machine on which it originates sends probe messages to a randomly-chosen machine, asking if its load is below some threshold value. If so, the process is sent there. If not, another machine is chosen for probing. Probing does not go on forever. If no suitable host is found within  $N$  probes, the algorithm terminates and the process runs on the originating machine.

An analytical queueing model of this algorithm has been constructed and investigated. Using this model, it was established that the algorithm behaves well and is stable under a wide range of parameters, including different threshold values, transfer costs, and probe limits.

Nevertheless, it should be observed that under conditions of heavy load, all machines will constantly send probes to other machines in a futile attempt to find one that is willing to accept more work. Few processes will be off-loaded, but considerable overhead may be incurred in the attempt to do so.

### **A Receiver-Initiated Distributed Heuristic Algorithm**

A complementary algorithm to the one given above, which is initiated by an overloaded sender, is one initiated by an underloaded receiver. With this algorithm, whenever a process finishes, the system checks to see if it has enough work. If not, it picks some machine at random and asks it for work. If that machine has nothing to offer, a second, and then a third machine is asked. If no work is found with  $N$  probes, the receiver temporarily stops asking, does any

work it has queued up, and tries again when the next process finishes. If no work is available, the machine goes idle. After some fixed time interval, it begins probing again.

An advantage of this algorithm is that it does not put extra load on the system at critical times. The sender-initiated algorithm makes large numbers of probes precisely when the system can least tolerate it—when it is heavily loaded. With the receiver-initiated algorithm, when the system is heavily loaded, the chance of a machine having insufficient work is small, but when this does happen, it will be easy to find work to take over. Of course, when there is little work to do, the receiver-initiated algorithm, creates considerable probe traffic as all the unemployed machines desperately hunt for work. However, it is far better to have the overhead go up when the system is underloaded than when it is overloaded.

It is also possible to combine both of these algorithms and have machines try to get rid of work when they have too much, and try to acquire work when they do not have enough. Furthermore, machines can perhaps improve on random polling by keeping a history of past probes to determine if any machines are chronically underloaded or overloaded. One of these can be tried first, depending on whether the initiator is trying to get rid of work or acquire it.

### **A Bidding Algorithm**

Another class of algorithms tries to turn the computer system into a miniature economy, with buyers and sellers of services and prices set by supply and demand (Ferguson et al., 1988). The key players in the economy are the processes, which must buy CPU time to get their work done, and processors, which auction their cycles off to the highest bidder.

Each processor advertises its approximate price by putting it in a publicly readable file. This price is not guaranteed, but gives an indication of what the service is worth (actually, it is the price that the last customer paid). Different processors may have different prices, depending on their speed, memory size, presence of floating-point hardware, and other features. An indication of the service provided, such as expected response time, can also be published.

When a process wants to start up a child process, it goes around and checks out who is currently offering the service that it needs. It then determines the set of processors whose services it can afford. From this set, it computes the best candidate, where “best” may mean cheapest, fastest, or best price/performance, depending on the application. It then generates a bid and sends the bid to its first choice. The bid may be higher or lower than the advertised price.

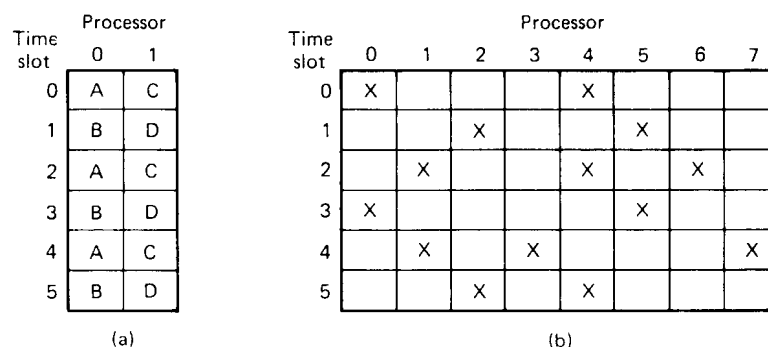
Processors collect all the bids sent to them, and make a choice, presumably by picking the highest one. The winners and losers are informed, and the

winning process is executed. The published price of the server is then updated to reflect the new going rate.

Although Ferguson et al. do not go into the details, such an economic model raises all kinds of interesting questions, among them the following. Where do processes get money to bid? Do they get regular salaries? Does everyone get the same monthly salary, or do deans get more than professors, who in turn get more than students? If new users are introduced into the system without a corresponding increase in resources, do prices get bid up (inflation)? Can processors form cartels to gouge users? Are users' unions allowed? Is disk space also chargeable? How about laser printer output? The list goes on and on.

#### 4.4. SCHEDULING IN DISTRIBUTED SYSTEMS

There is not really a lot to say about scheduling in a distributed system. Normally, each processor does its own local scheduling (assuming that it has multiple processes running on it), without regard to what the other processors are doing. Usually, this approach works fine. However, when a group of related, heavily interacting processes are all running on different processors, independent scheduling is not always the most efficient way.



**Fig. 4-20.** (a) Two jobs running out of phase with each other. (b) Scheduling matrix for eight processors, each with six time slots. The Xs indicated allocated slots.

The basic difficulty can be illustrated by an example in which processes *A* and *B* run on one processor and processes *C* and *D* run on another. Each processor is timeshared in, say, 100-msec time slices, with *A* and *C* running in the even slices and *B* and *D* running in the odd ones, as shown in Fig. 4-20(a). Suppose that *A* sends many messages or makes many remote procedure calls to *D*. During time slice 0, *A* starts up and immediately calls *D*, which unfortunate is not

running because it is now *C*'s turn. After 100 msec, process switching takes place, and *D* gets *A*'s message, carries out the work, and quickly replies. Because *B* is now running, it will be another 100 msec before *A* gets the reply and can proceed. The net result is one message exchange every 200 msec. What is needed is a way to ensure that processes that communicate frequently run simultaneously.

Although it is difficult to determine dynamically the interprocess communication patterns, in many cases, a group of related processes will be started off together. For example, it is usually a good bet that the filters in a UNIX pipeline will communicate with each other more than they will with other, previously started processes. Let us assume that processes are created in groups and that intragroup communication is much more prevalent than intergroup communication. Let us assume further that a sufficiently large number of processors is available to handle the largest group, and that each processor is multiprogrammed with  $N$  process slots ( $N$ -way multiprogramming).

Ousterhout (1982) proposed several algorithms based on a concept he calls **co-scheduling**, which takes interprocess communication patterns into account while scheduling to ensure that all members of a group run at the same time. The first algorithm uses a conceptual matrix in which each column is the process table for one processor, as shown in Fig. 4-20(b). Thus, column 4 consists of all the processes that run on processor 4. Row 3 is the collection of all processes that are in slot 3 of some processor, starting with the process in slot 3 of processor 0, then the process in slot 3 of processor 1, and so on. The gist of his idea is to have each processor use a round-robin scheduling algorithm with all processors first running the process in slot 0 for a fixed period, then all processors running the process in slot 1 for a fixed period, and so on. A broadcast message could be used to tell each processor when to do process switching, to keep the time slices synchronized.

By putting all the members of a process group in the same slot number, but on different processors, one has the advantage of  $N$ -fold parallelism, with a guarantee that all the processes will be run at the same time, to maximize communication throughput. Thus in Fig. 4-20(b), four processes that must communicate should be put into slot 3, on processors 1, 2, 3, and 4 for optimum performance. This scheduling technique can be combined with the hierarchical model of process management used in MICROS by having each department head maintain the matrix for its workers, assigning processes to slots in the matrix and broadcasting time signals.

Ousterhout also described several variations to this basic method to improve performance. One of these breaks the matrix into rows and concatenates the rows to form one long row. With  $k$  processors, any  $k$  consecutive slots belong to different processors. To allocate a new process group to slots, one lays a window  $k$  slots wide over the long row such that the leftmost slot is empty but the

slot just outside the left edge of the window is full. If sufficient empty slots are present in the window, the processes are assigned to the empty slots; otherwise, the window is slid to the right and the algorithm repeated. Scheduling is done by starting the window at the left edge and moving rightward by about one window's worth per time slice, taking care not to split groups over windows. Ousterhout's paper discusses these and other methods in more detail and gives some performance results.

## 4.5. FAULT TOLERANCE

A system is said to fail when it does not meet its specification. In some cases, such as a supermarket's distributed ordering system, a failure may result in some store running out of canned beans. In other cases, such in a distributed air traffic control system, a failure may be catastrophic. As computers and distributed systems become widely used in safety-critical missions, the need to prevent failures becomes correspondingly greater. In this section we will examine some issues concerning system failures and how they can be avoided. Additional introductory material can be found in (Cristian, 1991; and Nelson, 1990). Gantenbein (1992) has compiled a bibliography on the subject.

### 4.5.1. Component Faults

Computer systems can fail due to a fault in some component, such as a processor, memory, I/O device, cable, or software. A **fault** is a malfunction, possibly caused by a design error, a manufacturing error, a programming error, physical damage, deterioration in the course of time, harsh environmental conditions (it snowed on the computer), unexpected inputs, operator error, rodents eating part of it, and many other causes. Not all faults lead (immediately) to system failures, but some do.

Faults are generally classified as transient, intermittent, or permanent. **Transient faults** occur once and then disappear. If the operation is repeated, the fault goes away. A bird flying through the beam of a microwave transmitter may cause lost bits on some network (not to mention a roasted bird). If the transmission times out and is retried, it will probably work the second time.

An **intermittent fault** occurs, then vanishes of its own accord, then reappears, and so on. A loose contact on a connector will often cause an intermittent fault. Intermittent faults cause a great deal of aggravation because they are difficult to diagnose. Typically, whenever the fault doctor shows up, the system works perfectly.

A **permanent fault** is one that continues to exist until the faulty component



is repaired. Burnt-out chips, software bugs, and disk head crashes often cause permanent faults.

The goal of designing and building fault-tolerant systems is to ensure that the system as a whole continues to function correctly, even in the presence of faults. This aim is quite different from simply engineering the individual components to be highly reliable, but allowing (even expecting) the system to fail if one of the components does so.

Faults and failures can occur at all levels: transistors, chips, boards, processors, operating systems, user programs, and so on. Traditional work in the area of fault tolerance has been concerned mostly with the statistical analysis of electronic component faults. Very briefly, if some component has a probability  $p$  of malfunctioning in a given second of time, the probability of it *not* failing for  $k$  consecutive seconds and then failing is  $p(1-p)^k$ . The expected time to failure is then given by the formula

$$\text{mean time to failure} = \sum_{k=1}^{\infty} kp(1-p)^{k-1}$$

Using the well-known equation for an infinite sum starting at  $k=1$ :  $\sum \alpha^k = \alpha/(1-\alpha)$ , substituting  $\alpha = 1-p$ , differentiating both sides of the resulting equation with respect to  $p$ , and multiplying through by  $-p$  we see that

$$\text{mean time to failure} = 1/p$$

For example, if the probability of a crash is  $10^{-6}$  per second, the mean time to failure is  $10^6$  sec or about 11.6 days.

#### 4.5.2. System Failures

In a critical distributed system, often we are interested in making the *system* be able to survive component (in particular, processor) faults, rather than just making these unlikely. System reliability is especially important in a distributed system due to the large number of components present, hence the greater chance of one of them being faulty.

For the rest of this section, we will discuss processor faults or crashes, but this should be understood to mean equally well process faults or crashes (e.g., due to software bugs). Two types of processor faults can be distinguished:

1. Fail-silent faults.
2. Byzantine faults.

With **fail-silent faults**, a faulty processor just stops and does not respond to subsequent input or produce further output, except perhaps to announce that it is no

longer functioning. These are also called **fail-stop faults**. With **Byzantine faults**, a faulty processor continues to run, issuing wrong answers to questions, and possibly working together maliciously with other faulty processors to give the impression that they are all working correctly when they are not. Undetected software bugs often exhibit Byzantine faults. Clearly, dealing with Byzantine faults is going to be much more difficult than dealing with fail-silent ones.

The term “Byzantine” refers to the Byzantine Empire, a time (330-1453) and place (the Balkans and modern Turkey) in which endless conspiracies, intrigue, and untruthfulness were alleged to be common in ruling circles. Byzantine faults were first analyzed by Pease et al. (1980) and Lamport et al. (1982). Some researchers also consider combinations of these faults with communication line faults, but since standard protocols can recover from line errors in predictable ways, we will examine only processor faults.

#### 4.5.3. Synchronous versus Asynchronous Systems

As we have just seen, component faults can be transient, intermittent, or permanent, and system failures can be fail-silent or Byzantine. A third orthogonal axis deals with performance in a certain abstract sense. Suppose that we have a system in which if one processor sends a message to another, it is guaranteed to get a reply within a time  $T$  known in advance. Failure to get a reply means that the receiving system has crashed. The time  $T$  includes sufficient time to deal with lost messages (by sending them up to  $n$  times).

In the context of research on fault tolerance, a system that has the property of always responding to a message within a known finite bound if it is working is said to be **synchronous**. A system not having this property is said to be **asynchronous**. While this terminology is unfortunately, since it conflicts with more traditional uses of the terms, it is widely used among workers in fault tolerance.

It should be intuitively clear that asynchronous systems are going to be harder to deal with than synchronous ones. If a processor can send a message and know that the absence of a reply within  $T$  sec means that the intended recipient has failed, it can take corrective action. If there is no upper limit to how long the response might take, even determining whether there has been a failure is going to be a problem.

#### 4.5.4. Use of Redundancy

The general approach to fault tolerance is to use redundancy. Three kinds are possible: information redundancy, time redundancy, and physical redundancy. With information redundancy, extra bits are added to allow recovery

from garbled bits. For example, a Hamming code can be added to transmitted data to recover from noise on the transmission line.

With time redundancy, an action is performed, and then, if need be, it is performed again. Using the atomic transactions described in Chap. 3 is an example of this approach. If a transaction aborts, it can be redone with no harm. Time redundancy is especially helpful when the faults are transient or intermittent.

With physical redundancy, extra equipment is added to make it possible for the system as a whole to tolerate the loss or malfunctioning of some components. For example, extra processors can be added to the system so that if a few of them crash, the system can still function correctly.

There are two ways to organize these extra processors: active replication and primary backup. Consider the case of a server. When active replication is used, all the processors are used all the time as servers (in parallel) in order to hide faults completely. In contrast, the primary backup scheme just uses one processor as a server, replacing it with a backup if it fails.

We will discuss these two strategies below. For both of them, the issues are:

1. The degree of replication required.
2. The average and worst-case performance in the absence of faults.
3. The average and worst-case performance when a fault occurs.

Theoretical analyses of many fault-tolerant systems can be done in these terms. For more information, see (Schneider, 1990; and Budhiraja et al., 1993).

#### 4.5.5. Fault Tolerance Using Active Replication

**Active replication** is a well-known technique for providing fault tolerance using physical redundancy. It is used in biology (mammals have two eyes, two ears, two lungs, etc.), aircraft (747s have four engines but can fly on three), and sports (multiple referees in case one misses an event). Some authors refer to active replication as the **state machine approach**.

It has also been used for fault tolerance in electronic circuits for years. Consider, for example, the circuit of Fig. 4-21(a). Here signals pass through devices *A*, *B*, and *C*, in sequence. If one of them is faulty, the final result will probably be wrong.

In Fig. 4-21(b), each device is replicated three times. Following each stage in the circuit is a triplicated voter. Each voter is a circuit that has three inputs and one output. If two or three of the inputs are the same, the output is equal to that input. If all three inputs are different, the output is undefined. This kind of design is known as **TMR (Triple Modular Redundancy)**.

Suppose element  $A_2$  fails. Each of the voters,  $V_1$ ,  $V_2$ , and  $V_3$  gets two good

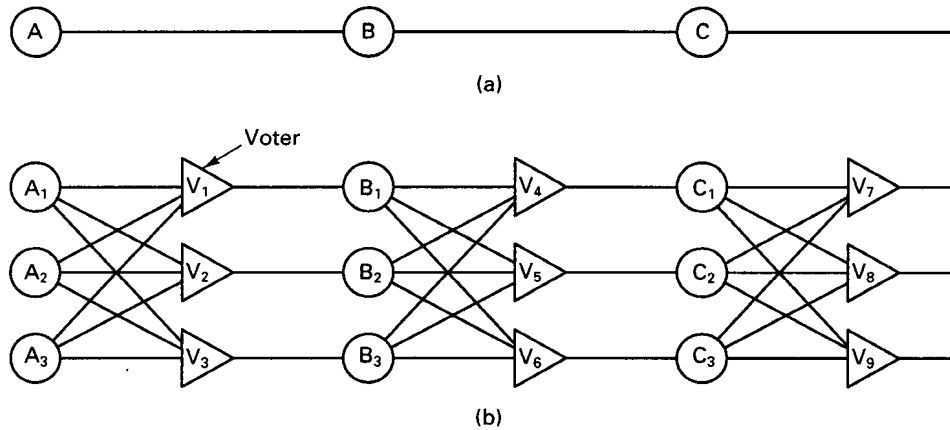


Fig. 4-21. Triple modular redundancy.

(identical) inputs and one rogue input, and each of them outputs the correct value to the second stage. In essence, the effect of  $A_2$  failing is completely masked, so that the inputs to  $B_1$ ,  $B_2$ , and  $B_3$  are exactly the same as they would have been had no fault occurred.

Now consider what happens if  $B_3$  and  $C_1$  are also faulty, in addition to  $A_2$ . These effects are also masked, so the three final outputs are still correct.

At first it may not be obvious why three voters are needed at each stage. After all, one voter could also detect and pass through the majority view. However, a voter is also a component and can also be faulty. Suppose, for example, that  $V_1$  malfunctions. The input to  $B_1$  will then be wrong, but as long as everything else works,  $B_2$  and  $B_3$  will produce the same output and  $V_4$ ,  $V_5$ , and  $V_6$  will all produce the correct result into stage three. A fault in  $V_1$  is effectively no different than a fault in  $B_1$ . In both cases  $B_1$  produces incorrect output, but in both cases it is voted down later.

Although not all fault-tolerant distributed operating systems use TMR, the technique is very general, and should give a clear feeling for what a fault-tolerant system is, as opposed to a system whose individual components are highly reliable but whose organization is not fault tolerant. Of course, TMR can be applied recursively, for example, to make a chip highly reliable by using TMR inside it, unknown to the designers who use the chip.

Getting back to fault tolerance in general and active replication in particular, in many systems, servers act like big finite-state machines: they accept requests and produce replies. Read requests do not change the state of the server, but write requests do. If each client request is sent to each server, and they all are received and processed in the same order, then after processing each one, all

nonfaulty servers will be in exactly the same state and will give the same replies. The client or voter can combine all the results to mask faults.

An important issue is how much replication is needed. The answer depends on the amount of fault tolerance desired. A system is said to be  **$k$  fault tolerant** if it can survive faults in  $k$  components and still meet its specifications. If the components, say processors, fail silently, then having  $k + 1$  of them is enough to provide  $k$  fault tolerance. If  $k$  of them simply stop, then the answer from the other one can be used.

On the other hand, if the processors exhibit Byzantine failures, continuing to run when sick and sending out erroneous or random replies, a minimum of  $2k + 1$  processors are needed to achieve  $k$  fault tolerance. In the worst case, the  $k$  failing processors could accidentally (or even intentionally) generate the same reply. However, the remaining  $k + 1$  will also produce the same answer, so the client or voter can just believe the majority.

Of course, in theory it is fine to say that a system is  $k$  fault tolerant and just let the  $k + 1$  identical replies outvote the  $k$  identical replies, but in practice it is hard to imagine circumstances in which one can say with certainty that  $k$  processors can fail but  $k + 1$  processors cannot fail. Thus even in a fault-tolerant system some kind of statistical analysis may be needed.

An implicit precondition for this finite state machine model to be relevant is that all requests arrive at all servers in the same order, sometimes called the **atomic broadcast problem**. Actually, this condition can be relaxed slightly, since reads do not matter and some writes may commute, but the general problem remains. One way to make sure that all requests are processed in the same order at all servers is to number them globally. Various protocols have been devised to accomplish this goal. For example, all requests could first be sent to a global number server to get a serial number, but then provision would have to be made for the failure of this server (e.g., by making it internally fault tolerant).

Another possibility is to use Lamport's logical clocks, as described in Chap. 3. If each message sent to a server is tagged with a timestamp, and servers process all requests in timestamp order, all requests will be processed in the same order at all servers. The trouble with this method is that when a server receives a request, it does not know whether any earlier requests are currently under way. In fact, most timestamp solutions suffer from this problem. In short, active replication is not a trivial matter. Schneider (1990) discusses the problems and solutions in some detail.

#### 4.5.6. Fault Tolerance Using Primary Backup

The essential idea of the primary-backup method is that at any one instant, one server is the primary and does all the work. If the primary fails, the backup takes over. Ideally, the cutover should take place in a clean way and be noticed

only by the client operating system, not by the application programs. Like active replication, this scheme is widely used in the world. Some examples are government (the Vice President), aviation (co-pilots), automobiles (spare tires), and diesel-powered electrical generators in hospital operating rooms.

Primary-backup fault tolerance has two major advantages over active replication. First, it is simpler during normal operation since messages go to just one server (the primary) and not to a whole group. The problems associated with ordering these messages also disappear. Second, in practice it requires fewer machines, because at any instant one primary and one backup is needed (although when a backup is put into service as a primary, a new backup is needed instantly). On the downside, it works poorly in the presence of Byzantine failures in which the primary erroneously claims to be working perfectly. Also, recovery from a primary failure can be complex and time consuming.

As an example of the primary-backup solution, consider the simple protocol of Fig. 4-22 in which a write operation is depicted. The client sends a message to the primary, which does the work and then sends an update message to the backup. When the backup gets the message, it does the work and then sends an acknowledgement back to the primary. When the acknowledgement arrives, the primary sends the reply to the client.

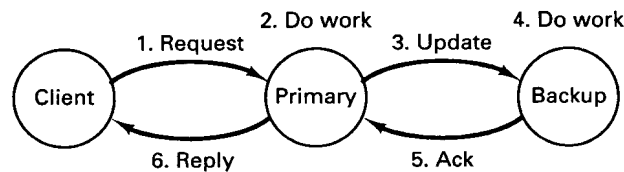


Fig. 4-22. A simple primary-backup protocol on a write operation.

Now let us consider the effect of a primary crash at various moments during an RPC. If the primary crashes before doing the work (step 2), no harm is done. The client will time out and retry. If it tries often enough, it will eventually get the backup and the work will be done exactly once. If the primary crashes after doing the work but before sending the update, when the backup takes over and the request comes in again, the work will be done a second time. If the work has side effects, this could be a problem. If the primary crashes after step 4 but before step 6, the work may end up being done three times, once by the primary, once by the backup as a result of step 3, and once after the backup becomes the primary. If requests carry identifiers, it may be possible to ensure that the work is done only twice, but getting it done exactly once is difficult to impossible.

One theoretical and practical problem with the primary-backup approach is when to cut over from the primary to the backup. In the protocol above, the backup could send: "Are you alive?" messages periodically to the primary. If the primary fails to respond within a certain time, the backup would take over.

However, what happens if the primary has not crashed, but is merely slow (i.e., we have an asynchronous system)? There is no way to distinguish between a slow primary and one that has gone down. Yet there is a need to make sure that when the backup takes over, the primary really stops trying to act like the primary. Ideally the backup and primary should have a protocol to discuss this, but it is hard to negotiate with the dead. The best solution is a hardware mechanism in which the backup can forcibly stop or reboot the primary. Note that all primary-backup schemes require agreement, which is tricky to achieve, whereas active replication does not always require an agreement protocol (e.g., TMR).

A variant of the approach of Fig. 4-22 uses a dual-ported disk shared between the primary and secondary. In this configuration, when the primary gets a request, it writes the request to disk before doing any work and also writes the results to disk. No messages to or from the backup are needed. If the primary crashes, the backup can see the state of the world by reading the disk. The disadvantage of this scheme is that there is only one disk, so if that fails, everything is lost. Of course, at the cost of extra equipment and performance, the disk could also be replicated and all writes could be done to both disks.

#### 4.5.7. Agreement in Faulty Systems

In many distributed systems there is a need to have processes agree on something. Examples are electing a coordinator, deciding whether to commit a transaction or not, dividing up tasks among workers, synchronization, and so on. When the communication and processors are all perfect, reaching such agreement is often straightforward, but when they are not, problems arise. In this section we will look at some of the problems and their solutions (or lack thereof).

The general goal of distributed agreement algorithms is to have all the non-faulty processors reach consensus on some issue, and do that within a finite number of steps. Different cases are possible depending on system parameters, including:

1. Are messages delivered reliably all the time?
2. Can processes crash, and if so, fail-silent or Byzantine?
3. Is the system synchronous or asynchronous?

Before considering the case of faulty processors, let us look at the “easy” case of perfect processors but communication lines that can lose messages. There is a famous problem, known as the **two-army problem**, which illustrates the difficulty of getting even two perfect processors to reach agreement about 1

bit of information. The red army, with 5000 troops, is encamped in a valley. Two blue armies, each 3000 strong, are encamped on the surrounding hillsides overlooking the valley. If the two blue armies can coordinate their attacks on the red army, they will be victorious. However, if either one attacks by itself it will be slaughtered. The goal of the blue armies is to reach agreement about attacking. The catch is that they can only communicate using an unreliable channel: sending a messenger who is subject to capture by the red army.

Suppose that the commander of blue army 1, General Alexander, sends a message to the commander of blue army 2, General Bonaparte, reading: "I have a plan—let's attack at dawn tomorrow." The messenger gets through and Bonaparte sends him back with a note saying: "Splendid idea, Alex. See you at dawn tomorrow." The messenger gets back to his base safely, delivers his messages, and Alexander tells his troops to prepare for battle at dawn.

However, later that day, Alexander realizes that Bonaparte does not know if the messenger got back safely and not knowing this, may not dare to attack. Consequently, Alexander tells the messenger to go tell Bonaparte that his (Bonaparte's) message arrived and that the battle is set.

Once again the messenger gets through and delivers the acknowledgement. But now Bonaparte worries that Alexander does not know if the acknowledgement got through. He reasons that if Bonaparte thinks that the messenger was captured, he will not be sure about his (Alexander's) plans, and may not risk the attack, so he sends the messenger back again.

Even if the messenger makes it through every time, it is easy to show that Alexander and Bonaparte will never reach agreement, no matter how many acknowledgements they send. Assume that there is some protocol that terminates in a finite number of steps. Remove any extra steps at the end to get the minimum protocol that works. Some message is now the last one and it is essential to the agreement (because this is the minimum protocol). If this message fails to arrive, the war is off.

However, the sender of the last message does not know if the last message arrived. If it did not, the protocol did not complete and the other general will not attack. Thus the sender of the last message cannot know if the war is scheduled or not, and hence cannot safely commit his troops. Since the receiver of the last message knows the sender cannot be sure, he will not risk certain death either, and there is no agreement. Even with nonfaulty processors (generals), agreement between even two processes is not possible in the face of unreliable communication.

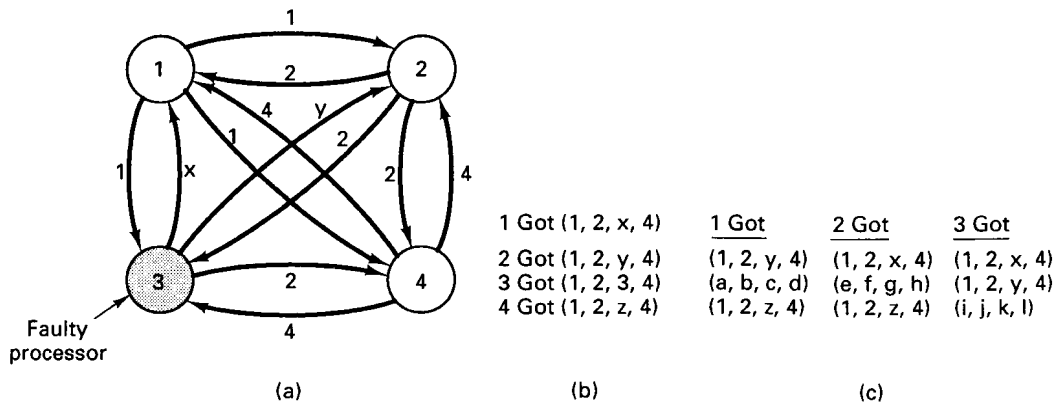
Now let us assume that the communication is perfect but the processors are not. The classical problem here also occurs in a military setting and is called the **Byzantine generals problem**. In this problem the red army is still encamped in the valley, but  $n$  blue generals all head armies on the nearby hills. Communication is done pairwise by telephone and is perfect, but  $m$  of the generals are



traitors (faulty) and are actively trying to prevent the loyal generals from reaching agreement by feeding them incorrect and contradictory information (to model malfunctioning processors). The question is now whether the loyal generals can still reach agreement.

For the sake of generality, we will define agreement in a slightly different way here. Each general is assumed to know how many troops he has. The goal of the problem is for the generals to exchange troop strengths, so that at the end of the algorithm, each general has a vector of length  $n$  corresponding to all the armies. If general  $i$  is loyal, then element  $i$  is his troop strength; otherwise, it is undefined.

A recursive algorithm was devised by Lamport et al. (1982) that solves this problem under certain conditions. In Fig. 4-23 we illustrate the working of the algorithm for the case of  $n = 4$  and  $m = 1$ . For these parameters, the algorithm operates in four steps. In step one, every general sends a (reliable) message to every other general announcing his troop strength. Loyal generals tell the truth; traitors may tell every other general a different lie. In Fig. 4-23(a) we see that general 1 reports 1K troops, general 2 reports 2K troops, general 3 lies to everyone, giving  $x$ ,  $y$ , and  $z$ , respectively, and general 4 reports 4K troops. In step 2, the results of the announcements of step 1 are collected together in the form of the vectors of Fig. 4-23(b).



**Fig. 4-23.** The Byzantine generals problem for 3 loyal generals and 1 traitor. (a) The generals announce their troop strengths (in units of 1K). (b) The vectors that each general assembles based on (a). (c) The vectors that each general receives in step 2.

Step 3 consists of every general passing his vector from Fig. 4-23(b) to every other general. Here, too, general 3 lies through his teeth, inventing 12 new values,  $a$  through  $l$ . The results of step 3 are shown in Fig. 4-23(c). Finally, in step 4, each general examines the  $i$ th element of each of the newly

received vectors. If any value has a majority, that value is put into the result vector. If no value has a majority, the corresponding element of the result vector is marked UNKNOWN. From Fig. 4-23(c) we see that generals 1, 2, and 4 all come to agreement on

(1, 2, UNKNOWN, 4)

which is the correct result. The traitor was not able to gum up the works.

Now let us revisit this problem for  $m = 3$  and  $n = 1$ , that is, only two loyal generals and one traitor, as illustrated in Fig. 4-24. Here we see that in Fig. 4-24(c) neither of the loyal generals sees a majority for element 1, element 2, or element 3, so all of them are marked UNKNOWN. The algorithm has failed to produce agreement.

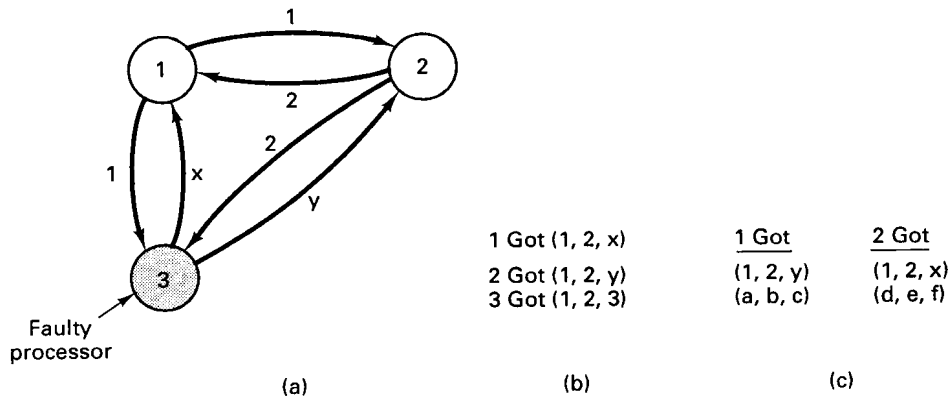


Fig. 4-24. The same as Fig. 4-23, except now with 2 loyal generals and one traitor.

In their paper, Lamport et al. (1982) proved that in a system with  $m$  faulty processors, agreement can be achieved only if  $2m + 1$  correctly functioning processors are present, for a total of  $3m + 1$ . Put in slightly different terms, agreement is possible only if *more* than two-thirds of the processors are working properly.

Worse yet, Fischer et al. (1985) proved that in a distributed system with asynchronous processors and unbounded transmission delays, no agreement is possible if even one processor is faulty (even if that one processor fails silently). The problem with asynchronous systems is that arbitrarily slow processors are indistinguishable from dead ones. Many other theoretical results are known about when agreement is possible and when it is not. Surveys of these results are given by Barborak et al. (1993) and Turek and Shasha (1992).

## 4.6. REAL-TIME DISTRIBUTED SYSTEMS

Fault-tolerant systems are not the only kind of specialized distributed systems. The real-time systems form another category. Sometimes these two are combined to give fault-tolerant real-time systems. In this section we will examine various aspects of real-time distributed systems. For additional material, see for example, (Burns and Wellings, 1990; Klein et al., 1994; and Shin, 1991).

### 4.6.1. What Is a Real-Time System?

For most programs, correctness depends only on the logical sequence of instructions executed, not when they are executed. If a C program correctly computes the double-precision floating-point square root function on a 200-MHz engineering workstation, it will also compute the function correctly on a 4.77-MHz 8088-based personal computer, only slower.

In contrast, **real-time programs** (and systems) interact with the external world in a way that involves time. When a stimulus appears, the system must respond to it in a certain way and before a certain deadline. If it delivers the correct answer, but after the deadline, the system is regarded as having failed. *When* the answer is produced is as important as *which* answer is produced.

Consider a simple example. An audio compact disk player consists of a CPU that takes the bits arriving from the disk and processes them to generate music. Suppose that the CPU is just barely fast enough to do the job. Now imagine that a competitor decides to build a cheaper player using a CPU running at one-third the speed. If it buffers all the incoming bits and plays them back at one-third the expected speed, people will wince at the sound, and if it only plays every third note, the audience will not be wildly ecstatic either. Unlike the earlier square root example, time is inherently part of the specification of correctness here.

Many other applications involving the external world are also inherently real time. Examples include computers embedded in television sets and video recorders, computers controlling aircraft ailerons and other parts (so called fly-by-wire), automobile subsystems controlled by computers (drive-by-wire?), military computers controlling guided antitank missiles (shoot-by-wire?), computerized air traffic control systems, scientific experiments ranging from particle accelerators to psychology lab mice with electrodes in their brains, automated factories, telephone switches, robots, medical intensive care units, CAT scanners, automatic stock trading systems, and numerous others.

Many real-time applications and systems are highly structured, much more so than general-purpose distributed systems. Typically, an external device (possibly a clock) generates a stimulus for the computer, which must then perform

certain actions before a deadline. When the required work has been completed, the system becomes idle until the next stimulus arrives.

Frequently, the stimuli are **periodic**, with a stimulus occurring regularly every  $\Delta T$  seconds, such as a computer in a TV set or VCR getting a new frame every  $1/60$  of a second. Sometimes stimuli are **aperiodic**, meaning that they are recurrent, but not regular, as in the arrival of an aircraft in a air traffic controller's air space. Finally, some stimuli are **sporadic** (unexpected), such as a device overheating.

Even in a largely periodic system, a complication is that there may be many types of events, such as video input, audio input, and motor drive management, each with its own period and required actions. Figure 4-25 depicts a situation with three periodic event streams, *A*, *B*, and *C*, plus one sporadic event, *X*.

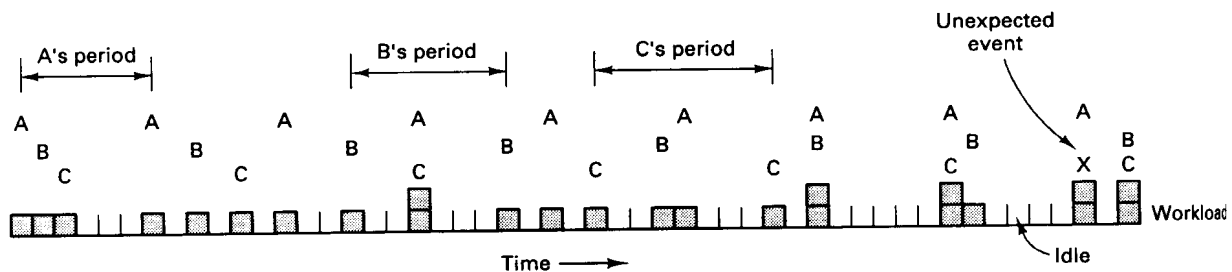


Fig. 4-25. Superposition of three event streams plus one sporadic event.

Despite the fact that the CPU may have to deal with multiple event streams, it is not acceptable for it to say: It is true that I missed event *B*, but it is not my fault—I was still working on *A* when *B* happened. While it is not hard to manage two or three input streams with priority interrupts, as applications get larger and more complex (e.g., automated factory assembly lines with thousands of robots), it will become more and more difficult for one machine to meet all the deadlines and other real-time constraints.

Consequently, some designers are experimenting with the idea of putting a dedicated microprocessor in front of each real-time device to accept output from it whenever it has something to say, and give it input at whatever speed it requires. Of course, this does not make the real-time character go away, but instead gives rise to a distributed real-time system, with its own unique characteristics and challenges (e.g., real-time communication).

Distributed real-time systems can often be structured as illustrated in Fig. 4-26. Here we see a collection of computers connected by a network. Some of these are connected to external devices that produce or accept data or expect to be controlled in real time. The computers may be tiny microcontrollers built into the devices, or stand-alone machines. In both cases they usually

have sensors for receiving signals from the devices and/or actuators for sending signals to them. The sensors and actuators may be digital or analog.

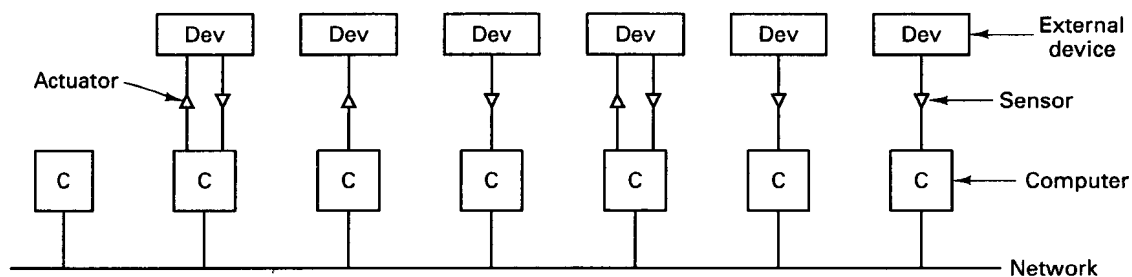


Fig. 4-26. A distributed real-time computer system.

Real-time systems are generally split into two types depending on how serious their deadlines are and the consequences of missing one. These are:

1. Soft real-time systems.
2. Hard real-time systems.

**Soft real-time** means that missing an occasional deadline is all right. For example, a telephone switch might be permitted to lose or misroute one call in  $10^5$  under overload conditions and still be within specification. In contrast, even a single missed deadline in a **hard real-time** system is unacceptable, as this might lead to loss of life or an environmental catastrophe. In practice, there are also intermediate systems where missing a deadline means you have to kill off the current activity, but the consequence is not fatal. For example, if a soda bottle on a conveyor belt has passed by the nozzle, there is no point in continuing to squirt soda at it, but the results are not fatal. Also, in some real-time systems, some subsystems are hard real time whereas others are soft real time.

Real-time systems have been around for decades, so there is a considerable amount of folk wisdom accumulated about them, most of it wrong. Stankovic (1988) has pointed out some of these myths, the worst of which are summarized here.

*Myth 1: Real-time systems are about writing device drivers in assembly code.*

This was perhaps true in the 1970s for real-time systems consisting of a few instruments attached to a minicomputer, but current real-time systems are too complicated to trust to assembly language and writing the device drivers is the least of a real-time system designer's worries.

*Myth 2: Real-time computing is fast computing.*

Not necessarily. A computer-controlled telescope may have to track stars or galaxies in real time, but the apparent rotation of the heavens is only 15 degrees of arc per hour of time, not especially fast. Here accuracy is what counts.

*Myth 3: Fast computers will make real-time system obsolete.*

No. They just encourage people to build real-time systems that were previously beyond the state-of-the-art. Cardiologists would love to have an MRI scanner that shows a beating heart inside an exercising patient in real time. When they get that, they will ask for it in three dimensions, in full color, and with the possibility of zooming in and out. Furthermore, making systems faster by using multiple processors introduces new communication, synchronization, and scheduling problems that have to be solved.

#### 4.6.2. Design Issues

Real-time distributed systems have some unique design issues. In this section we will examine some of the most important ones.

##### Clock Synchronization

The first issue is the maintenance of time itself. With multiple computers, each having its own local clock, keeping the clocks in synchrony is a key issue. We examined this point in Chap. 3, so we will not repeat that discussion here.

##### Event-Triggered versus Time-Triggered Systems

In an **event-triggered real-time system**, when a significant event in the outside world happens, it is detected by some sensor, which then causes the attached CPU to get an interrupt. Event-triggered systems are thus interrupt driven. Most real-time systems work this way. For soft real-time systems with lots of computing power to spare, this approach is simple, works well, and is still widely used. Even for more complex systems, it works well if the compiler can analyze the program and know all there is to know about the system behavior once an event happens, even if it cannot tell when the event will happen.

The main problem with event-triggered systems is that they can fail under conditions of heavy load, that is, when many events are happening at once. Consider, for example, what happens when a pipe ruptures in a computer-controlled nuclear reactor. Temperature alarms, pressure alarms, radioactivity alarms, and other alarms will all go off at once, causing massive interrupts. This **event shower** may overwhelm the computing system and bring it down,

potentially causing problems far more serious than the rupture of a single pipe.

An alternative design that does not suffer from this problem is the **time-triggered real-time system**. In this kind of system, a clock interrupt occurs every  $\Delta T$  milliseconds. At each clock tick (selected) sensors are sampled and (certain) actuators are driven. No interrupts occur other than clock ticks.

In the ruptured pipe example given above, the system would become aware of the problem at the first clock tick after the event, but the interrupt load would not change on account of the problem, so the system would not become overloaded. Being able to operate normally in times of crisis increases the chances of dealing successfully with the crisis.

It goes without saying that  $\Delta T$  must be chosen with extreme care. If it is too small, the system will get many clock interrupts and waste too much time fielding them. If it is too large, serious events may not be noticed until it is too late. Also, the decision about which sensors to check on every clock tick, and which to check on every other clock tick, and so on, is critical. Finally, some events may be shorter than a clock tick, so they must be saved to avoid losing them. They can be preserved electrically by latch circuits or by microprocessors embedded in the external devices.

As an example of the difference between these two approaches, consider the design of an elevator controller in a 100-story building. Suppose that the elevator is sitting peacefully on the 60th floor waiting for customers. Then someone pushes the call button on the first floor. Just 100 msec later, someone else pushes the call button on the 100th floor. In an event-triggered system, the first call generates an interrupt, which causes the elevator to take off downward. The second call comes in after the decision to go down has already been made, so it is noted for future reference, but the elevator continues on down.

Now consider a time-triggered elevator controller that samples every 500 msec. If both calls fall within one sampling period, the controller will have to make a decision, for example, using the nearest-customer-first rule, in which case it will go up.

In summary, event-triggered designs give faster response at low load but more overhead and chance of failure at high load. Time-trigger designs have the opposite properties and are furthermore only suitable in a relatively static environment in which a great deal is known about system behavior in advance. Which one is better depends on the application. In any event, we note that there is much lively controversy over this subject in real-time circles.

### Predictability

One of the most important properties of any real-time system is that its behavior be predictable. Ideally, it should be clear at design time that the system can meet all of its deadlines, even at peak load. Statistical analyses of

behavior assuming independent events are often misleading because there may be unsuspected correlations between events, as between the temperature, pressure, and radioactivity alarms in the ruptured pipe example above.

Most distributed system designers are used to thinking in terms of independent users accessing shared files at random or numerous travel agents accessing a shared airline data base at unpredictable times. Fortunately, this kind of chance behavior rarely holds in a real-time system. More often, it is known that when event *E* is detected, process *X* should be run, followed by processes *Y* and *Z*, in either order or in parallel. Furthermore, it is often known (or should be known) what the worst-case behavior of these processes is. For example, if it is known that *X* needs 50 msec, *Y* and *Z* need 60 msec each, and process startup takes 5 msec, then it can be guaranteed in advance that the system can flawlessly handle five periodic type *E* events per second in the absence of any other work. This kind of reasoning and modeling leads to a deterministic rather than a stochastic system.

### **Fault Tolerance**

Many real-time systems control safety-critical devices in vehicles, hospitals, and power plants, so fault tolerance is frequently an issue. Active replication is sometimes used, but only if it can be done without extensive (and thus time-consuming) protocols to get everyone to agree on everything all the time. Primary-backup schemes are less popular because deadlines may be missed during cutover after the primary fails. A hybrid approach is to follow the leader, in which one machine makes all the decisions, but the others just do what it says to do without discussion, ready to take over at a moment's notice.

In a safety-critical system, it is especially important that the system be able to handle the worst-case scenario. It is not enough to say that the probability of three components failing at once is so low that it can be ignored. Failures are not always independent. For example, during a sudden electric power failure, everyone grabs the telephone, possibly causing the phone system to overload, even though it has its own independent power generation system. Furthermore, the peak load on the system often occurs precisely at the moment when the maximum number of components have failed because much of the traffic is related to reporting the failures. Consequently, fault-tolerant real-time systems must be able to cope with the maximum number of faults and the maximum load at the same time.

Some real-time systems have the property that they can be stopped cold when a serious failure occurs. For instance, when a railroad signaling system unexpectedly blacks out, it may be possible for the control system to tell every train to stop immediately. If the system design always spaces trains far enough apart and all trains start braking more-or-less simultaneously, it will be possible



to avert disaster and the system can recover gradually when the power comes back on. A system that can halt operation like this without danger is said to be **fail-safe**.

### Language Support

While many real-time systems and applications are programmed in general-purpose languages such as C, specialized real-time languages can potentially be of great assistance. For example, in such a language, it should be easy to express the work as a collection of short tasks (e.g., lightweight processes or threads) that can be scheduled independently, subject to user-defined precedence and mutual exclusion constraints.

The language should be designed so that the maximum execution time of every task can be computed at compile time. This requirement means that the language cannot support general **while** loops. Iteration must be done using **for** loops with constant parameters. Recursion cannot be tolerated either (it is beginning to look like FORTRAN has a use after all). Even these restrictions may not be enough to make it possible to calculate the execution time of each task in advance since cache misses, page faults, and cycle stealing by DMA channels all affect performance, but they are a start.

Real-time languages need a way to deal with time itself. To start with, a special variable, *clock*, should be available, containing the current time in ticks. However, one has to be careful about the unit that time is expressed in. The finer the resolution, the faster *clock* will overflow. If it is a 32-bit integer, for example, the range for various resolutions is shown in Fig. 4-27. Ideally, the clock should be 64 bits wide and have a 1 nsec resolution.

| Clock resolution | Range      |
|------------------|------------|
| 1 nsec           | 4 seconds  |
| 1 $\mu$ sec      | 72 minutes |
| 1 msec           | 50 days    |
| 1 sec            | 136 years  |

Fig. 4-27. Range of a 32-bit clock before overflowing for various resolutions.

The language should have a way to express minimum and maximum delays. In Ada<sup>®</sup>, for example, there is a delay statement that specifies a minimum value that a process must be suspended. However, the actual delay may be more by an unbounded amount. There is no way to give an upper bound or a time interval in which the delay is required to fall.

There should also be a way to express what to do if an expected event does not occur within a certain interval. For example, if a process blocks on a semaphore for more than a certain time, it should be possible to time out and be released. Similarly, if a message is sent, but no reply is forthcoming fast enough, the sender should be able to specify that it is to be deblocked after  $k$  msec.

Finally, since periodic events play such a big role in real-time systems, it would be useful to have a statement of the form

```
every (25 msec) { ... }
```

that causes the statements within the curly brackets to be executed every 25 msec. Better yet, if a task contains several such statements, the compiler should be able to compute what percentage of the CPU time is required by each one, and from these data compute the minimum number of machines needed to run the entire program and how to assign processes to machines.

#### 4.6.3. Real-Time Communication

Communication in real-time distributed systems is different from communication in other distributed systems. While high performance is always welcome, predictability and determinism are the real keys to success. In this section we will look at some real-time communication issues, for both LANs and WANs. Finally, we will examine one example system in some detail to show how it differs from conventional (i.e., non-real-time) distributed systems. Alternative approaches are described in (Malcolm and Zhao, 1994; and Ramanathan and Shin, 1992)

Achieving predictability in a distributed system means that communication between processors must also be predictable. LAN protocols that are inherently stochastic, such as Ethernet, are unacceptable because they do not provide a known upper bound on transmission time. A machine wanting to send a packet on an Ethernet may collide with one or more other machines. All machines then wait a random time and then try again, but these transmissions may also collide, and so on. Consequently, it is not possible to give a worst-case bound on packet transmission in advance.

As a contrast to Ethernet, consider a token ring LAN. Whenever a processor has a packet to send, it waits for the circulating token to pass by, then it captures the token, sends its packet, and puts the token back on the ring so that the next machine downstream gets the opportunity to seize it. Assuming that each of the  $k$  machines on the ring is allowed to send at most one  $n$ -byte packet per token capture, it can be guaranteed that an urgent packet arriving anywhere in the system can always be transmitted within  $kn$  byte times. This is the kind of upper bound that a real-time distributed system needs.

Token rings can also handle traffic consisting of multiple priority classes. The goal here is to ensure that if a high-priority packet is waiting for transmission, it will be sent before any low-priority packets that its neighbors may have. For example, it is possible to add a reservation field to each packet, which can be increased by any processor as the packet goes by. When the packet has gone all the way around, the reservation field indicates the priority class of the next packet. When the current sender is finished transmitting, it regenerates a token bearing this priority class. Only processors with a pending packet of this class may capture it, and then only to send one packet. Of course, this scheme means that the upper bound of  $kn$  byte times now applies only to packets of the highest priority class.

An alternative to a token ring is the **TDMA (Time Division Multiple Access)** protocol shown in Fig. 4-28. Here traffic is organized in fixed-size frames, each of which contains  $n$  slots. Each slot is assigned to one processor, which may use it to transmit a packet when its time comes. In this way collisions are avoided, the delay is bounded, and each processor gets a guaranteed fraction of the bandwidth, depending on how many slots per frame it has been assigned.

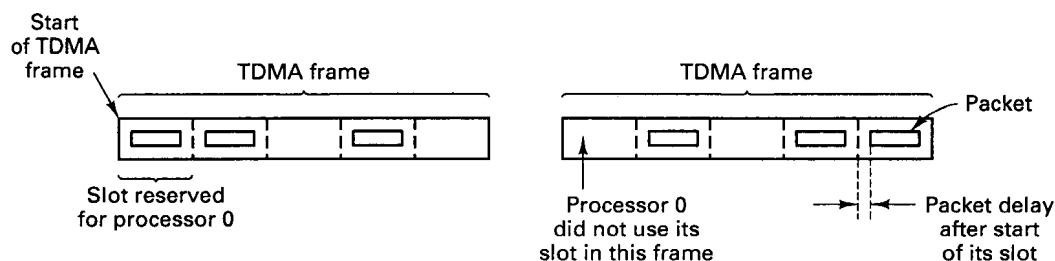


Fig. 4-28. TDMA (Time Division Multiple Access) frames.

Real-time distributed systems operating over wide-area networks have the same need for predictability as those confined to a room or building. The communication in these systems is invariably connection oriented. Often, there is the ability to establish **real-time connections** between distant machines. When such a connection is established, the quality of service is negotiated in advance between the network users and the network provider. This quality may involve a guaranteed maximum delay, maximum jitter (variance of packet delivery times), minimum bandwidth, and other parameters. To make good on its guarantees, the network may have to reserve memory buffers, table entries, CPU cycles, link capacity, and other resources for this connection throughout its lifetime. The user is likely to be charged for these resources, whether or not they are used, since they are not available to other connections.

A potential problem with wide-area real-time distributed systems is their

relatively high packet loss rates. Standard protocols deal with packet loss by setting a timer when each packet is transmitted. If the timer goes off before the acknowledgement is received, the packet is sent again. In real-time systems, this kind of unbounded transmission delay is rarely acceptable.

One easy solution is for the sender *always* to transmit each packet two (or more) times, preferably over independent connections if that option is available. Although this scheme wastes at least half the bandwidth, if one packet in, say,  $10^5$  is lost, only one time in  $10^{10}$  will both copies be lost. If a packet takes a millisecond, this works out to one lost packet every four months. With three transmissions, one packet is lost every 30,000 years. The net effect of multiple transmissions of every packet right from the start is a low and bounded delay virtually all the time.

### The Time-Triggered Protocol

On account of the constraints on real-time distributed systems, their protocols are often quite unusual. In this section we will examine one such protocol, **TTP (Time-Triggered Protocol)** (Kopetz and Grunsteidl, 1994), which is as different from the Ethernet protocol as a Victorian drawing room is from a Wild West saloon. TTP is used in the MARS real-time system (Kopetz et al., 1989) and is intertwined with it in many ways, so we will refer to properties of MARS where necessary.

A node in MARS consists of at least one CPU, but often two or three work together to present the image of a single fault-tolerant, fail-silent node to the outside world. The nodes in MARS are connected by two reliable and independent TDMA broadcast networks. All packets are sent on both networks in parallel. The expected loss rate is one packet every 30 million years.

MARS is a time-triggered system, so clock synchronization is critical. Time is discrete, with clock ticks generally occurring every microsecond. TTP assumes that all the clocks are synchronized with a precision on the order of tens of microseconds. This precision is possible because the protocol itself provides continuous clock synchronization and has been designed to allow it to be done in hardware to extremely high precision.

All nodes in MARS are aware of the programs being run on all the other nodes. In particular, all nodes know when a packet is to be sent by another node and can detect its presence or absence easily. Since packets are assumed not to be lost (see above), the absence of a packet at a moment when one is expected means that the sending node has crashed.

For example, suppose that some exceptional event is detected and a packet is broadcast to tell everyone else about it. Node 6 is expected to make some computation and then broadcast a reply after 2 msec in slot 15 of the TDMA frame. If the message is not forthcoming in the expected slot, the other nodes

assume that node 6 has gone down, and take whatever steps are necessary to recover from its failure. This tight bound and instant consensus eliminate the need for time-consuming agreement protocols and allow the system to be both fault tolerant and operate in real time.

Every node maintains the global state of the system. These states are required to be identical everywhere. It is a serious (and detectable) error if someone is out of step with the rest. The global state consists of three components:

1. The current mode.
2. The global time.
3. A bit-map giving the current system membership.

The mode is defined by the application and has to do with which phase the system is in. For example, in a space application, the countdown, launch, flight, and landing might all be separate modes. Each mode has its own set of processes and the order in which they run, list of participating nodes, TDMA slot assignments, message names and formats, and legal successor modes.

The second field in the global state is the global time. Its granularity is application defined, but in any event must be coarse enough that all nodes agree on it. The third field keeps track of which nodes are up and which are down.

Unlike the OSI and Internet protocol suites, the TTP protocol consists of a single layer that handles end-to-end data transport, clock synchronization, and membership management. A typical packet format is illustrated in Fig. 4-29. It consists of a start-of-packet field, a control field, a data field, and a CRC field.

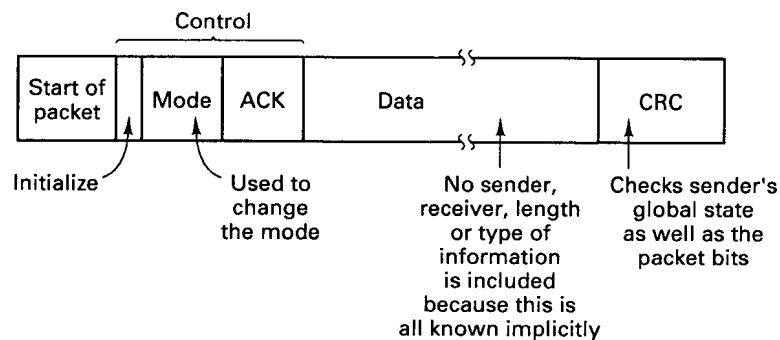


Fig. 4-29. A typical TTP packet.

The control field contains a bit used to initialize the system (more about which later), a subfield for changing the current mode, and a subfield for acknowledging the packets sent by the preceding node (according to the current

membership list). The purpose of this field is to let the previous node know that it is functioning correctly and its packets are getting onto the network as they should be. If an expected acknowledgement is lacking, all nodes mark the expected sender as down and expunge it from the membership bit maps in their current state. The rejected node is expected to go along with being excommunicated without protest.

The data field contains whatever data are required. The CRC field is quite unusual, as it provides a checksum over not only the packet contents, but over the sender's global state as well. This means that if a sender has an incorrect global state, the CRC of any packets it sends will not agree with the values the receivers compute using their states. The next sender will not acknowledge the packet, and all nodes, including the one with the bad state, mark it as down in their membership bit maps.

Periodically, a packet with the initialization bit is broadcast. This packet also contains the current global state. Any node that is marked as not being a member, but which is supposed to be a member in this mode, can now join as a passive member. If a node is supposed to be a member, it has a TDMA slot assigned, so there is no problem of when to respond (in its own TDMA slot). Once its packet has been acknowledged, all the other nodes mark it as being active (operational) again.

A final interesting aspect of the protocol is the way it handles clock synchronization. Because each node knows the time when TDMA frames start and the position of its slot within the frame, it knows exactly when to begin its packet. This scheme avoids collisions. However, it also contains valuable timing information. If a packet begins  $n$  microseconds before or after it is supposed to, each other node can detect this tardiness and use it as an estimate of the skew between its clock and the sender's clock. By monitoring the starting position of every packet, a node might learn, for example, that every other node appears to be starting its transmissions 10 microseconds too late. In this case it can reasonably conclude that its own clock is actually 10 microseconds fast and make the necessary correction. By keeping a running average of the earliness or lateness of all other packets, each node can adjust its clock continuously to keep it in sync with the others without running any special clock management protocol.

In summary, the unusual properties of TTP are the detection of lost packets by the receivers, not the senders, the automatic membership protocol, the CRC on the packet plus global state, and the way that clock synchronization is done.

#### 4.6.4. Real-Time Scheduling

Real-time systems are frequently programmed as a collection of short tasks (processes or threads), each with a well-defined function and a well-bounded execution time. The response to a given stimulus may require multiple tasks to

be run, generally with constraints on their execution order. In addition, a decision has to be made about which tasks to run on which processors. In this section we will deal with some of the issues concerning task scheduling in real-time systems.

Real-time scheduling algorithms can be characterized by the following parameters:

1. Hard real time versus soft real time.
2. Preemptive versus nonpreemptive scheduling.
3. Dynamic versus static.
4. Centralized versus decentralized.

Hard real-time algorithms must guarantee that all deadlines are met. Soft real-time algorithms can live with a best efforts approach. The most important case is hard real time.

Preemptive scheduling allows a task to be suspended temporarily when a higher-priority task arrives, resuming it later when no higher-priority tasks are available to run. Nonpreemptive scheduling runs each task to completion. Once a task is started, it continues to hold its processor until it is done. Both kinds of scheduling strategies are used.

Dynamic algorithms make their scheduling decisions during execution. When an event is detected, a dynamic preemptive algorithm decides on the spot whether to run the (first) task associated with the event or to continue running the current task. A dynamic nonpreemptive algorithm just notes that another task is runnable. When the current task finishes, a choice among the now-ready tasks is made.

With static algorithms, in contrast, the scheduling decisions, whether preemptive or not, are made in advance, before execution. When an event occurs, the runtime scheduler just looks in a table to see what to do.

Finally, scheduling can be centralized, with one machine collecting all the information and making all the decisions, or it can be decentralized, with each processor making its own decisions. In the centralized case, the assignment of tasks to processors can be made at the same time. In the decentralized case, assigning tasks to processors is distinct from deciding which of the tasks assigned to a given processor to run next.

A key question that all real-time system designers face is whether or not it is even possible to meet all the constraints. If a system has one processor and it gets 60 interrupts/sec, each requiring 50 msec of work, the designers have a Big Problem on their hands.

Suppose that a periodic real-time distributed system has  $m$  tasks and  $N$  processors to run them on. Let  $C_i$  be the CPU time needed by task  $i$ , and let  $P_i$  be

its period, that is, the time between consecutive interrupts. To be feasible, the utilization of the system,  $\mu$ , must be related to  $N$  by the equation

$$\mu = \sum_{i=1}^m \frac{C_i}{P_i} \leq N$$

For example, if a task is started every 20 msec and runs for 10 msec each time, it uses up 0.5 CPUs. Five such tasks would need three CPUs to do the job. A set of tasks that meets the foregoing requirement is said to be **schedulable**. Note that the equation above really gives a lower bound on the number of CPUs needed, since it ignores task switching time, message transport, and other sources of overhead, and assumes that optimal scheduling is possible.

In the following two sections we will look at dynamic and static scheduling, respectively, of sets of periodic tasks. For additional information, see (Ramamirtham et al., 1990; and Schwan and Zhou, 1992).

### Dynamic Scheduling

Let us look first at a few of the better-known dynamic scheduling algorithms—algorithms that decide during program execution which task to run next. The classic algorithm is the **rate monotonic algorithm** (Liu and Layland, 1973). It was designed for preemptively scheduling periodic tasks with no ordering or mutual exclusion constraints on a single processor. It works like this. In advance, each task is assigned a priority equal to its execution frequency. For example, a task run every 20 msec is assigned priority 50 and a task run every 100 msec is assigned priority 10. At run time, the scheduler always selects the highest priority task to run, preempting the current task if need be. Liu and Layland proved that this algorithm is optimal. They also proved that any set of tasks meeting the utilization condition

$$\mu = \sum_{i=1}^m \frac{C_i}{P_i} \leq m(2^{1/m} - 1)$$

is schedulable using the rate monotonic algorithm. The right-hand side converges to  $\ln 2$  (about 0.693) as  $m \rightarrow \infty$ . In practice, this limit is overly pessimistic; a set of tasks with  $\mu$  as high as 0.88 can usually be scheduled.

A second popular preemptive dynamic algorithm is **earliest deadline first**. Whenever an event is detected, the scheduler adds it to the list of waiting tasks. This list is always kept sorted by deadline, closest deadline first. (For a periodic task, the deadline is the next occurrence.) The scheduler then just chooses the first task on the list, the one closest to its deadline. Like the rate monotonic algorithm, it produces optimal results, even for task sets with  $\mu = 1$ .



A third preemptive dynamic algorithm first computes for each task the amount of time it has to spare, called the **laxity** (slack). For a task that must finish in 200 msec but has another 150 msec to run, the laxity is 50 msec. This algorithm, called **least laxity**, chooses the task with the least laxity, that is, the one with the least breathing room.

None of the algorithms above are provably optimal in a distributed system, but they can be used as heuristics. Also, none of them takes into account order or mutex constraints, even on a uniprocessor, which makes them less useful in practice than they are in theory. Consequently, many practical systems use static scheduling when enough information is available. Not only can static algorithms take side constraints into account, but they have very low overhead at run time.

### Static Scheduling

Static scheduling is done before the system starts operating. The input consists of a list of all the tasks and the times that each must run. The goal is to find an assignment of tasks to processors and for each processor, a static schedule giving the order in which the tasks are to be run. In theory, the scheduling algorithm can run an exhaustive search to find the optimal solution, but the search time is exponential in the number of tasks (Ullman, 1976), so heuristics of the type described above are generally used. Rather than simply give some additional heuristics here, we will go into one example in detail, to show the interplay of scheduling and communication in a real-time distributed system with nonpreemptive static scheduling (Kopetz et al., 1989).

Let us assume that every time a certain event is detected, task 1 is started on processor *A*, as shown in Fig. 4-30. This task, in turn, starts up additional tasks, both locally and on a second processor, *B*. For simplicity, we will assume that the assignment of tasks to processors is dictated by external considerations (which task needs access to which I/O device) and is not a parameter here. All tasks take 1 unit of CPU time.

Task 1 starts up tasks 2 and 3 on its machine, as well as task 7 on processor *B*. Each of these three tasks starts up another task, and so on, as illustrated. The arrows indicate messages being sent between tasks. In this simple example, it is perhaps easiest to think of  $X \rightarrow Y$  as meaning that *Y* cannot start until a message from *X* has arrived. Some tasks, such as 8, require two messages before they may start. The cycle is completed when task 10 has run and generated the expected response to the initial stimulus.

After task 1 has completed, tasks 2 and 3 are both runnable. The scheduler has a choice of which one to schedule next. Suppose that it decides to schedule task 2 next. It then has a choice between tasks 3 and 4 as the successor to task 2. If it chooses task 3, it then has a choice between tasks 4 and 5 next. However, if

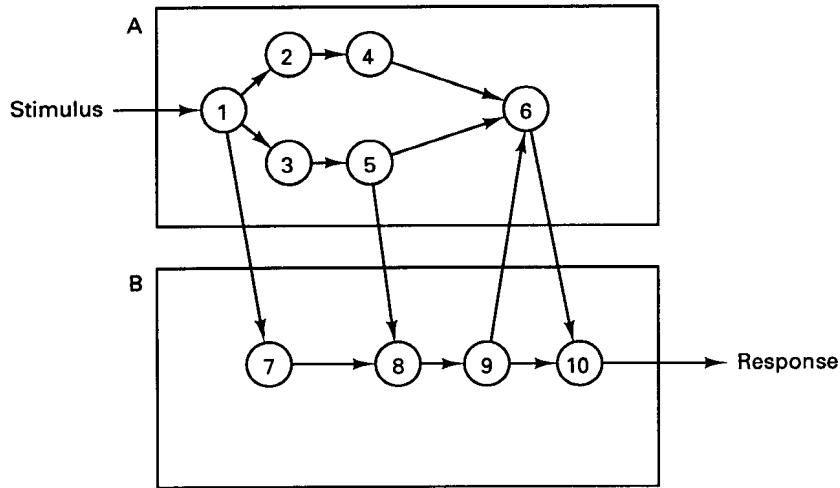


Fig. 4-30. Ten real-time tasks to be executed on two processors.

it chooses 4 instead of 3, it must run 3 following 4, because 6 is not enabled yet, and will not be until both 5 and 9 have run.

Meanwhile, activity is also occurring in parallel on processor *B*. As soon as task 1 has initiated it, task 7 can start on *B*, at the same time as either 2 or 3. When both 5 and 7 have finished, task 8 can be started, and so on. Note that task 6 requires input from 4, 5, and 9 to start, and produces output for 10.

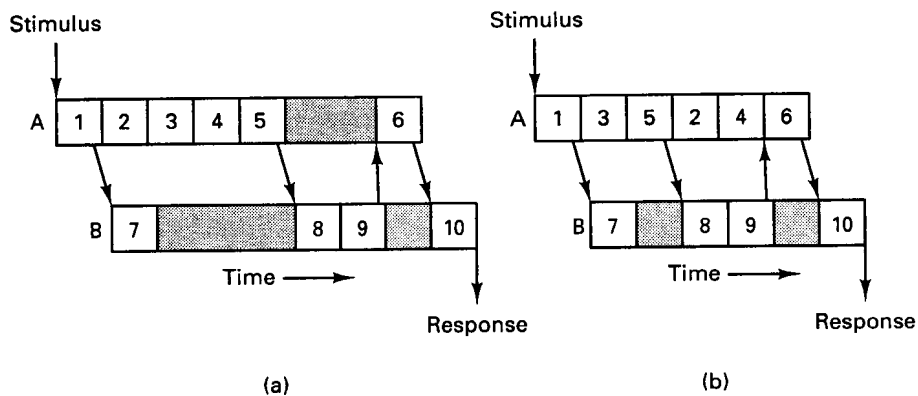


Fig. 4-31. Two possible schedules for the tasks of Fig. 4-30.

Two potential schedules are given in Fig. 4-31(a) and (b). Messages between tasks on different processors are depicted as arrows here; messages between tasks on the same machine are handled internally and are not shown. Of the two schedules illustrated, the one in Fig. 4-31(b) is a better choice

because it allows task 5 to run early, thus making it possible for task 8 to start earlier. If task 5 is delayed significantly, as in Fig. 4-31(a), then tasks 8 and 9 are delayed, which also means that 6 and eventually 10 are delayed, too.

It is important to realize that with static scheduling, the decision of whether to use one of these schedules, or one of several alternatives is made by the scheduler *in advance*, before the system starts running. It analyzes the graph of Fig. 4-30, also using as input information about the running times of all the tasks, and then applies some heuristic to find a good schedule. Once a schedule has been selected, the choices made are incorporated into tables so that at run time a simple dispatcher can carry out the schedule with little overhead.

Now let us consider the problem of scheduling the same tasks again, but this time taking communication into account. We will use TDMA communication, with eight slots per TDMA frame. In this example, a TDMA slot is equal to one-fourth of a task execution time. We will arbitrarily assign slot 1 to processor A and slot 5 to processor B. The assignment of TDMA slots to processors is up to the static scheduler and may differ between program phases.

In Fig. 4-32 we show both schedules of Fig. 4-31, but now taking the use of TDMA slots into account. A task may not send a message until a slot owned by its processor comes up. Thus, task 5 may not send to task 8 until the first slot of the next TDMA frame occurs in rotation, requiring a delay in starting task 8 in Fig. 4-32(a) that was not present before.

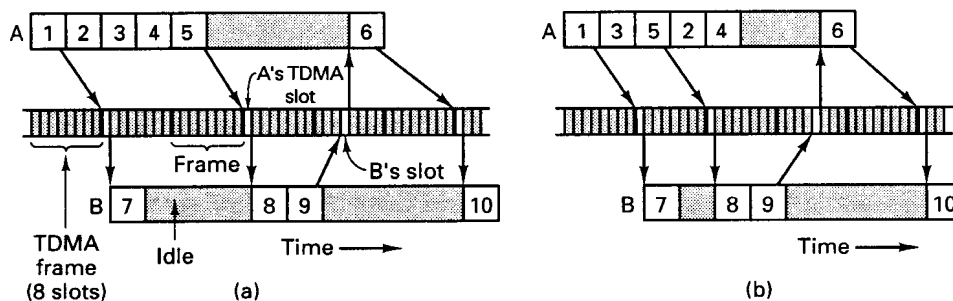


Fig. 4-32. Two schedules, including processing and communication.

The important thing to notice about this example is that the runtime behavior is completely deterministic, and known even before the program starts executing. As long as communication and processor errors do not occur, the system will always meet its real-time deadlines. Processor failures can be masked by having each node consist of two or more CPUs actively tracking each other. Some extra time may have to be statically allocated to each task interval to allow for recovery, if need be. Lost or garbled packets can be handled by having every one sent twice initially, either on disjoint networks or on one network by making the TDMA slots two packets wide.

It should be clear by now that real-time systems do not try to squeeze the last drop of performance out of the hardware, but rather use extra resources to make sure that the real-time constraints are met under all conditions. However, the relatively low use of the communication bandwidth in our example is not typical. It is a consequence of this example using only two processors with modest communication requirements. Practical real-time systems have many processors and extensive communication.

### **A Comparison of Dynamic versus Static Scheduling**

The choice of dynamic or static scheduling is an important one and has far-reaching consequences for the system. Static scheduling is a good fit with a time-triggered design, and dynamic scheduling is a good fit for an event-triggered design. Static scheduling must be carefully planned in advance, with considerable effort going into choosing the various parameters. Dynamic scheduling does not require as much advance work, since scheduling decisions are made on-the-fly, during execution.

Dynamic scheduling can potentially make better use of resources than static scheduling. In the latter, the system must frequently be overdimensioned to have so much capacity that it can handle even the most unlikely cases. However, in a hard real-time system, wasting resources is often the price that must be paid to guarantee that all deadlines will be met.

On the other hand, given enough computing power, an optimal or nearly optimal schedule can be derived in advance for a static system. For an application such as reactor control, it may well be worth investing months of CPU time to find the best schedule. A dynamic system cannot afford the luxury of a complex scheduling calculation during execution, so to be safe, may have to be heavily overdimensioned as well, and even then, there is no guarantee that it will meet its specifications. Instead, extensive testing is required.

As a final thought, it should be pointed out that our discussion has simplified matters considerably. For example, tasks may need access to shared variables, so these have to be reserved in advance. Often there are scheduling constraints, which we have ignored. Finally, some systems do advance planning during run-time, making them hybrids between static and dynamic.

## **4.7. SUMMARY**

Although threads are not an inherent feature of distributed operating systems, most of them have a threads package, so we have studied them in this chapter. A thread is a kind of lightweight process that shares an address space with one or more other threads. Each thread has its own program counter and